mercoledì 22 ottobre 2025

Lezione 1 - Introduzione e Motivazioni

Il Calcolo ad Alte Prestazioni

Il *calcolo ad alte prestazioni* (HPC) rappresenta l'evoluzione moderna del supercalcolo (*supercomputing*).
È una disciplina che utilizza algoritmi, software e soprattutto hardware avanzato (supercomputer) per risolvere problemi su larga scala con grande rapidità e precisione.

Il supercalcolo trova applicazione in settori come:

- Meteorologia e previsioni climatiche,
- Sicurezza informatica,
- Sanità e bioinformatica,
- Trasporti, pubblicità, finanza, media, educazione, ecc.

Tutti questi ambiti condividono la necessità di ottenere una **soluzione "in tempo reale" o "in tempo utile"**.

Supercomputer e misura delle prestazioni

Definizione di Supercomputer

Un **supercomputer** è un sistema di calcolo che, in un determinato momento storico, offre **le prestazioni più elevate disponibili**.
Le sue prestazioni vengono valutate in termini di **tempo necessario a risolvere una particolare applicazione**, quindi sono *application-dependent*.

Classifica mondiale – TOP500

Dal 1993, due volte l'anno, viene pubblicata la lista **TOP500** dei supercomputer più veloci del mondo.
La misura di riferimento è:

$$R_{max}$$

che rappresenta la **performance massima** ottenuta con il **benchmark LINPACK**, ossia un test standardizzato che misura la velocità nel risolvere un sistema di equazioni lineari $Ax = b$

- A: matrice dei coefficienti (dimensione n
- x: vettore incognito
- b: vettore dei termini noti
- **Scopo:** trovare $\mathrm{x\ tale\ che}\ A\,x = b$.

  Il LINPACK misura quanti *floating point operations per second* (**FLOPS**) il computer riesce a eseguire durante questa risoluzione.

Rmax e HPL

Il valore R_{max} è calcolato tramite **HPL (High Performance Linpack)**, una versione parallela e portabile del benchmark.

In Italia, il riferimento è il **supercomputer Leonardo** del CINECA (Bologna), in grado di eseguire **milioni di miliardi di operazioni al secondo** (petaflop/s).

Motivazioni: perché serve il calcolo parallelo

Un Software è un programma che codifica una procedura seriale in una serie discreta di istruzioni eseguite in successione.

La lezione propone un'analogia per spiegare che **non sempre serve il massimo delle prestazioni**, ma esse diventano cruciali **a seconda dell'obiettivo**:

- In città: una Panda (meno potente) è sufficiente.
- In autostrada: serve la potenza e l'affidabilità di una Ferrari.

Allo stesso modo, un supercomputer è necessario **solo** per problemi che richiedono tempi di risposta molto rapidi o trattano dati enormi.

Esempio 1 – Ricerca su Internet

Ogni giorno milioni di persone interrogano un motore di ricerca.
Il tempo di attesa accettabile per una risposta è **inferiore a 1 secondo** ($\tau < 1s$).
Questo impone un'enorme potenza di calcolo distribuita.

Esempio 2 – Difesa e sistemi militari

Storicamente, il supercalcolo nasce per **scopi bellici** ed esigenze economiche.
Durante la **Guerra del Golfo**, il calcolo parallelo guidava i missili Patriot contro gli Scud iracheni:

- tempo utile per reagire: $\tau < 2\mathrm{minuti}$

Come ridurre il tempo di esecuzione di un software

Il **tempo di esecuzione complessivo** di un programma è modellato da:

$$\tau = k \cdot T(N) \cdot \mu$$

- τ: tempo totale di esecuzione del software;
- k: costante moltiplicativa aleatoria che rappresenta tempi casuali di I/O o ritardi di scrittura in memoria;
- $T(N)$: complessità computazionale dell'algoritmo (dipendente dal metodo risolutivo scelto);
- μ:tempo medio necessario per eseguire un'operazione elementare (floating point, di conseguenza dipende all'ambiente di calcolo).

Strategie di riduzione di riduzione di τ

1. **Ridurre** $\mu \rightarrow$ migliorando la **tecnologia hardware** (miniaturizzazione, CPU più rapide).
    - Limiti: *surriscaldamento* e barriere fisiche.
2. **Ridurre** $T(N) \rightarrow$ migliorando l'**algoritmo**.
    - *Teoria della complessità* mostra che esistono limiti teorici: non tutti i problemi possono avere algoritmi più veloci degli ottimali.
3. **Ridurre** $k \rightarrow$ cambiando paradigma computazionale, es. **calcolo quantistico**, che modifica la rappresentazione numerica dei dati e può consentire accessi più rapidi alla memoria. (Quindi computer con diversa rappresentazione numerica nella memorizzazione dei dati).
4. **Usare più unità di calcolo (**np**)** $\rightarrow$ introducendo il **calcolo parallelo:**

$$\tau = k \cdot T_{np}(N) \cdot \mu$$

dove np è il numero di unità processanti che lavorano contemporaneamente.

Il Calcolo Parallelo: concetto chiave

Il calcio parallelo è una evoluzione del calcolo seriale che tante di emulare ciò che spesso avviene nel mondo naturale: molteplici eventi complessi e inter-correlati che avvengono nello stesso tempo.

Il calcolo seriale differisce dal calcolo parallelo siccome tratta la soluzione di più problemi tramite procedure, quello parallelo invece risolve un unico problema spezzandolo in più parti discrete elaborabili.

L'idea del **calcolo parallelo** si basa sull'uso simultaneo di più risorse di calcolo per risolvere un unico problema.
L'idea è **suddividere il problema in sottoproblemi indipendenti**, elaborati contemporaneamente da più CPU.

Lezione 2 - Forme di Parallelismo e Classificazione di Flynn

Concetto operativo del calcolo parallelo

Il **calcolo parallelo** utilizza *più risorse di calcolo simultaneamente* per risolvere un singolo problema.
L'idea centrale è:

- **decomporre** un problema di dimensione N in np sottoproblemi di dimensione $\frac{N}{np}$
- eseguirli contemporaneamente su np CPU.
- **ricombinare i risultati** per ottenere la soluzione globale.

Questo approccio richiede:

- hardware e software dedicati,
- algoritmi progettati specificamente per l'ambiente parallelo.

Dalla modellizzazione alla progettazione parallela

Ogni problema reale deve essere tradotto in una catena coerente di rappresentazioni:

Fase	Descrizione
Modello numerico ($M_h(P)$)	rappresenta il problema matematico di partenza
Modello numerico parallelo ($M_{h,np}(P)$)	versione adattata all'ambiente con più processori
Algoritmo parallelo ($A_{np}(P)$)	insieme di istruzioni pensato per l'esecuzione su architettura parallela

Software parallelo ($S_{np}(P)$)	implementazione concreta del codice eseguibile

Dove il modello numerico viene implementato in algoritmo parallelo e software parallelo.

Fondamentale è considerare **le caratteristiche hardware** disponibili (numero di CPU, tipo di memoria, topologia di connessione).

Architetture hardware: dalla macchina di Von Neumann al parallelismo

Per approcciarsi al calcolo parallelo è necessaria una visione chiara di come è composta la macchina. Questa visione è data dalla CLASSIFICAZIONE DI FLYNN. La quale si basa sulla molteplicità di flusso di istruzioni e sul flusso di dati che il sistema può eseguire.

Architettura di Von Neumann (1945)

Schema classico di calcolatore:

- Unità di input/output
- Memoria
- Una Unità di controllo
- Una Unità logico-aritmetica (ALU)

Dati e istruzioni viaggiano sullo stesso bus.
È una macchina **sequenziale**: un'istruzione alla volta, su un singolo flusso di dati.

Che prende il nome di SISD(Single Instruction, Single Data), una sola unità di elaborazione che esegue una sola istruzione su un dato alla volta.

Con l'evoluzione tecnologica si sono introdotte forme di **parallelismo interno alla CPU**, dette *on-chip (anche perché l'implementazione avviene a livello hardware)*.

Flynn ne distinse varie architetture di calcolo, individuate dalla tassonomia di Flynn. Ne esistono tre principali, due che lavorano su un singolo chip:

1. Parallelismo temporale (MISD)
2. Parallelismo spaziale (SIMD)

A queste si aggiunge una terza forma più ampia che lavora su più chip: **parallelismo asincrono (MIMD)**.

Quindi in totale abbiamo 4 categorie: SISD, MISD,SIDM, MIMD.

Primo tipo di parallelismo: Parallelismo Temporale (Pipeline)

Definizione

Hardware = più unità funzionali all'interno di un'unica ALU.

Consiste nel suddividere un'operazione complessa in **più fasi sequenziali**, ciascuna gestita da una diversa *unità funzionale* dell'ALU.
Le fasi operano **contemporaneamente** su dati diversi.

Questa tipo di architettura prende il nome di MISD(Multiple Instruction, Single Data). Piu unita di elaborazione che eseguono istruzioni sullo stesso dato.

Esempi

Esempio tecnico: somma di 100 numeri floating point

Operazione:

$$i = 1,...,100$$

$$c_i = a_i + b_i$$

L'ALU tradizionale gestisce ogni somma una alla volta. Una **ALU pipelined** divide ogni somma $c_i = a_i + b_i$ in 4 fasi:

1. confronto degli esponenti,
2. shift delle mantisse,
3. somma,
4. normalizzazione.

Mentre il primo dato passa alla fase 2, il secondo entra nella fase 1 → tutte le fasi lavorano in parallelo.

Esecuzione tradizionale (Unità funzionale divisa in 4 segmenti) solo uno alla volta è attivo:

Tempo	Fase attiva	Dato
1	F1	(a_1,b_1)
2	F2	(a_1,b_1)
3	F3	(a_1,b_1)

4	F4	(a_1,b_1)
5	F1	(a_2,b_2)

$$T = N \cdot t_u \cdot 4$$

Esecuzione pipelined:

Tempo	F1	F2	F3	F4
1	(a_1,b_1)	—	—	—
2	(a_2,b_2)	(a_1,b_1)	—	—
3	(a_3,b_3)	(a_2,b_2)	(a_1,b_1)	—
4	(a_4,b_4)	(a_3,b_3)	(a_2,b_2)	(a_1,b_1)
5	(a_5,b_5)	(a_4,b_4)	(a_3,b_3)	(a_2,b_2)

$$T = [4 + (N-1)] \cdot t_u < 4Nt_u$$

Risparmio significativo: a regime, tutte le unità lavorano contemporaneamente.

Questa struttura è alla base dei **processori vettoriali**, efficienti per dati organizzati in vettori (es. operazioni su array).

Secondo tipo di parallelismo: Parallelismo Spaziale

Definizione

Consiste nel disporre più unità aritmetico-logiche (ALU) che eseguono la stessa istruzione su dati diversi, sotto un unico controllo (CU).

È un parallelismo “spaziale” perché più unità operano nello stesso istante su zone differenti dei dati.

Prende il nome di SIMD (Single Instruction, Multiple Data), o parallelismo Sincrono essendo su una sola CU. Parallelismo su dati una sola unità di elaborazione che esegue la stessa istruzione su più dati contemporaneamente.

Esempio

Tre operai eseguono contemporaneamente la *stessa azione* (es. posare mattoni) su *parti diverse* della casa.

Tempo (t_u)	Istruzione eseguita	ALU 1 (dato)	ALU 2 (dato)	ALU 3 (dato)	ALU 4 (dato)	Risultati prodotti
t_1	`LOAD A, B`	carica A_1, B_1	carica A_2, B_2	carica A_3, B_3	carica A_4, B_4	—
t_2	`ADD`	$A_1 + B_1$	$A_2 + B_2$	$A_3 + B_3$	$A_4 + B_4$	C_1, C_2, C_3, C_4
t_3	`STORE C`	salva C_1	salva C_2	salva C_3	salva C_4	(completato)

Terzo tipo di parallelismo: Parallelismo Asincrono

Definizione

Qui più CPU complete (ognuna con la propria CU e ALU) eseguono istruzioni diverse su dati diversi contemporaneamente.

Ogni processore è indipendente ma cooperante.

Prende il nome di MIMD (Multiple Instruction, Multiple Data), è il paradigma più generale e rappresenta i moderni sistemi multicore e cluster. Parallelismo su istruzioni e dati piu unita di elaborazione eseguono piu istruzioni su piu dati.

Esempio

Tre operai eseguono compiti diversi su parti diverse del lavoro (uno costruisce, uno dipinge, uno installa).

Tassonomia di Flynn (1965)

Sigla	Nome completo	Descrizione	Esempio
SISD	Single Instruction Single Data	Architettura sequenziale classica (Von Neumann)	PC tradizionale
MISD	Multiple Instruction Single Data	Monoprocessore con pipeline o unità funzionali multiple	Processori vettoriali particolari
SIMD	Single Instruction Multiple Data	Un'unica istruzione applicata a molti dati in parallelo	GPU, array processors
MIMD	Multiple Instruction Multiple Data	Più CPU indipendenti su dati diversi	Cluster, sistemi multicore

Architetture MIMD: memoria e interconnessione

Flynn distingue due grandi categorie nelle architettura MIMD:

Due principali categorie

- **Shared Memory (memoria condivisa)**
  I calcolatori hanno più CPU connesse e tutte le CPU accedono alla stessa memoria fisica.
  → Esempio: *Symmetric MultiProcessor* (SMP), come i moderni multicore Intel.
- **Distributed Memory (memoria distribuita)**
  I calcolatori hanno più CPU e ogni CPU ha la propria memoria locale.
  → Esempio: cluster o supercomputer collegati in rete.

Topologie di connessione

Nei sistemi **MIMD**, CPU e memorie possono essere collegate in diversi modi a seconda del tipo di problema e applicazione della macchina:

In un calcolatore **MIMD SM(Shared Memory)** la memoria e la CPU possono essere collegate con una di questi 3 modi**:**

- Bus singolo (tutte le CPU condividono un unico canale)
- Bus multiplo (maggiore banda)
- Connessione diretta

In un calcolatore **Distributed Memory (DM)** la memoria e la CPU possono essere collegate anche con schemi di connessione più complessi, come**:**

Bus, anello, reticolo 2D, albero, ipercubo, stella, switch.

Estensioni moderne della classificazione

MIMD DM:

- **Massive Parallel Processor:** Architettura con CPU in PC indipendenti che lavorano in parallelo(IBM RS/6000, Blue Gene, ASCI Red).
- **Distributed Shared Memory:** Architettura con CPU in PC indipendenti che lavorano in parallelo, in cui uno dei PC ha una memoria virtuale condivisa(CRAY T3E).

MIMD SM:

- **Symmetric MultiProcessor:** Architettura multicore con più core identici in un'unica memoria principale condivisa(es. Intel multicore).
- **ASymmetric MultiProcessor:** core eterogenei con funzioni diverse in un'unica macchina(es. PlayStation 5 con CPU+GPU).

GPU e architetture eterogenee

Le **GPU** (Graphic Processing Units) rappresentano oggi la forma più sofisticata di macchina parallela, in un normale processore multicore abbiamo per ogni core 1 CU + 1 ALU, in una GPU:

- Ogni "core" della GPU possiede una CU e un stream di ALU ("un insieme di ALU").
- Sono specializzate nell'eseguire milioni di operazioni(semplici) identiche su grandi insiemi di dati (es. pixel, neuroni, particelle).

Per questo oggi le GPU sono fondamentali non solo per la grafica, ma anche per *AI, simulazioni fisiche, deep learning, bioinformatica* ecc.

Cluster multiprocessori (MIMD-DM)

Un cluster è un insieme di calcolatori autonomi collegati tramite rete, ciascuno con propria CPU e memoria.
Può essere:

- **Omogeneo** (nodi identici),
- **Eterogeneo** (hardware differente).

Richiede software di coordinamento e comunicazione:
MPI (Message Passing Interface), nata nel 1991, standard fondamentale per il calcolo distribuito. Ogni processore ha una propria memoria locale alla quale

accede direttamente e può conoscere i dati in memoria di un'altra processore o far conoscere propri attraverso il trasferimento di dati.

Calcolo Parallelo vs Calcolo Distribuito

- **Calcolo parallelo:** ridurre il *wall-clock time*, cioè il tempo reale di esecuzione di un problema complesso.
- **Calcolo distribuito:** sfruttare in modo efficiente risorse geograficamente distribuite, massimizzando la disponibilità e minimizzando i costi.

Un **ambiente di calcolo parallelo** è un sistema di unità processanti strettamente collegate che comunicano per risolvere problemi su larga scala in maniera efficiente.

Un **ambiente di calcolo distribuito** è un sistema di unità processanti autonome e indipendenti, fisicamente distribuite che comunicano per risolvere problemi su larga scala in maniera efficiente.

Quindi sia il calcolo parallelo che il calcolo distribuito servono per risolvere problemi grandi più velocemente di quanto farebbe un singolo computer. Ma la differenza principale sta in come i vari processori sono collegati e qual'è lo scopo primario del sistema.

Con il calcolo parallelo l'obbiettivo sono le performance, e infatti si cerca di ridurre il tempo necessario alla risoluzione computazionale di un problema reale.

Con il calcolo distribuito l'obbiettivo è il riuso delle risorse esistenti, e infatti si cerca di riutilizzare "efficacemente" le risorse hardware e software distribuite geograficamente sul territorio.

Lezione 3 - Software Parallelo: Introduzione multicore e OpenMP

Architetture MIMD: Shared vs Distributed Memory

Due categorie principali

Tipo di architettura	Caratteristica	Esempio tipico	Problemi principali
MIMD – Shared Memory (SM)	Tutte le CPU accedono alla **stessa area di memoria condivisa**	Sistemi **multicore**	Necessità di **sincronizzare** gli accessi e organizzare il lavoro

MIMD – Distributed Memory (DM)	Ogni CPU possiede la **propria memoria locale**	**Cluster** di calcolatori	Necessità di **comunicare esplicitamente** tra processi (es. MPI)

Riepilogo concettuale

- **Shared Memory**: comunicazione implicita (variabili condivise).
- **Distributed Memory**: comunicazione esplicita (invio e ricezione messaggi).

La lezione si focalizza sul **caso Shared Memory (MIMD-SM)**, ossia **sistemi multicore**, dove più *core* condividono la stessa memoria fisica.

Thread e processi

Definizioni di base

- Un **processo** è un programma in esecuzione.
- Un **thread** (filo di esecuzione) è un *flusso indipendente di istruzioni* che viene eseguito sequenzialmente da una CPU.

Ogni processo contiene **almeno un thread**, ma può contenerne più di uno (multithreading).

Esecuzione multipla

- Thread diversi possono essere eseguiti:
  - su **CPU distinte** (ambiente MIMD-DM),
  - o su **core diversi** dello stesso chip (ambiente MIMD-SM, multicore).

Nel secondo caso, i thread di uno stesso processo **condividono la stessa area di memoria**: questo consente un'elevata efficienza ma richiede **sincronizzazione accurata** per evitare conflitti di accesso.

Architettura multicore

Nei sistemi **multicore**, più **core** all'interno dello stesso processore lavorano insieme, condividendo:

- la memoria principale,
- le cache di livello superiore,
- e parte del bus di comunicazione.

Vantaggi e Svantaggi

Vantaggi

- Maggiore efficienza e leggerezza rispetto a processi distinti.

Svantaggi

- Dati non protetti: l'accesso contemporaneo può causare **race conditions**, richiedendo **meccanismi di sincronizzazione**.

Memoria e variabili

In un processo multi-thread:

- Ogni thread ha **variabili private**;
- Alcune variabili sono **shared**, accessibili da tutti i thread.

I thread **comunicano implicitamente** attraverso queste variabili condivise, ed è necessario **coordinare l'accesso in scrittura e lettura**.

Strumenti per la programmazione in MIMD-SM

Tra le principali librerie:

- OpenMP (Open Multi-Processing)
- Pthreads (POSIX threads)
- Windows threads

Il corso si concentra su **OpenMP**, la più diffusa per semplicità e portabilità

Introduzione a OpenMP

Caratteristiche generali

- Specifiche **aperte** per la programmazione **multi-threaded** in memoria condivisa.
- Offre una API (Application Programming Interface) compatibile con C/C++ e Fortran, su sistemi Unix/Linux e Windows.
- Consente un approccio **user-friendly e portabile**, ideale per convertire un codice sequenziale in parallelo.

Modello di esecuzione: Fork-Join

1. **Start:** Il programma inizia con un solo *master thread* che esegue in modo sequenziale.
2. **Fork:** Quando il codice incontra una *regione parallela*, viene creato un *team di thread* che lavorano contemporaneamente.
3. **Join:** Alla fine della sezione parallela, tutti i thread si **sincronizzano** e terminano, lasciando proseguire solo il master.

Nota importante

Ogni thread mantiene una propria **cache locale**, ma la **coerenza della memoria non è automatica**: il programmatore deve scegliere con cura quali variabili rendere *private* o *shared*.

Struttura generale di un programma OpenMP

Codice

```
#include <omp.h>

main() {
   int var1, var2, var3;
   /* Parte sequenziale */

   #pragma omp parallel private(var1, var2) shared(var3)
   {
      /* Sezione parallela eseguita da tutti i thread */
   }

   /* Ripresa sequenziale */
}
```

Spiegazione

- La direttiva #pragma omp parallel crea un **team di thread**.
- Le **clausole** specificano la visibilità delle variabili:
  - `private(var1, var2)` → ogni thread ha la propria copia;
  - `shared(var3)` → variabile condivisa.
- Alla fine della sezione, i thread confluiscono nel *master thread*.

Direttive del compilatore

Le **direttive OpenMP** sono istruzioni speciali per il compilatore che si usano per creare un team e stabilire quali istruzioni devono essere eseguite in parallelo e come

devono essere distribuite tra i thread del team creato (possono essere innestate per parallelismo multilivello)

Codice

```
#pragma omp parallel [clause], [clause] ...
{
   structured-block
}
```

Alla fine del blocco è **sottintesa una barriera di sincronizzazione**: tutti i thread devono completare prima di proseguire.

Distribuzione del lavoro

Dopo la creazione dei thread, il programmatore deve **definire la ripartizione dei compiti** per evitare duplicazioni o conflitti.

Clausole

Le **clausole** sono opzioni aggiuntive delle direttive OpenMP (`parallel`, `for`, ecc.) che controllano il data sharing e il comportamento dei thread, specificano dettagli sul comportamento dei thread e sulla visibilità delle variabili.

Clausole Principali

Clausola	Descrizione	Effetto
`num_threads(int)`	Specifica il numero di thread del team	Imposta dimensione del team
default(shared|none)	Shared: Tutte le variabili saranno considerate condivise, None: sceglie il programmatore	Imposta la modalità di condivisione predefinita
`private(list)`	Ogni thread ha una copia privata	Valori indefiniti all'ingresso
`firstprivate(list)`	Come `private`, ma con inizializzazione dal valore originale	Le copie vengono inizializzate
`lastprivate(list)`	Gli argomenti contenuto in list sono privati per i thread e quelli originali verranno aggiornati al termine della regione parallela	Usata con costrutto `for`
`shared(list)`	Variabili visibili a tutti i thread	Comunicazione implicita

`reduction(operator:list)`	Combina risultati parziali dei thread usando l'operatore specificato	Somme, prodotti, min/max ecc.

Nota: il costrutto `reduction` non garantisce ordine d'esecuzione → nei calcoli in *floating point*, i risultati possono variare numericamente.

Dopo ogni costrutto `parallel`, `for`, `sections`, ecc. c'è una **barriera implicita**: i thread si fermano e attendono che tutti abbiano terminato, prima di proseguire.

Non tutte le clausole sono valide per tutti i costrutti: alcune (come `lastprivate`) funzionano solo con `for`, altre (come `num_threads`) solo con `parallel`.

Routine di libreria (Runtime Library Routines)

Per gestire i parametri a run-time è necessario includere **`omp.h`**.

Si accede a una serie di **funzioni C standardizzate** definite da OpenMP.
Esse permettono di:

- controllare il comportamento del *team di thread* durante l'esecuzione,
- ottenere informazioni sull'ambiente di parallelismo.

Queste funzioni **non** sono direttive (`#pragma`), ma **vere e proprie chiamate di libreria.**

Funzione	Descrizione
`omp_set_num_threads(int)`	imposta il numero di thread da usare
`omp_get_max_threads()`	restituisce il numero massimo disponibile
`omp_set_dynamic(int)`	abilita/disabilita l'adattamento dinamico
`omp_get_thread_num()`	restituisce l'ID del thread corrente
`omp_get_num_procs()`	numero di core disponibili
`omp_get_num_threads()`	numero di thread attivi nel team

Variabili d'ambiente

Nel sistema operativo, una variabile d'ambiente è una coppia `NOME=VALORE` visibile da tutti i processi.
Serve per configurare il comportamento di programmi e librerie *senza modificare il codice sorgente*.

OpenMP utilizza alcune variabili d'ambiente ad esempio per:

- impostare il numero di thread,

- abilitare o disabilitare **l'adattamento dinamico**,

Consentono di impostare valori *prima* dell'esecuzione:

- `OMP_NUM_THREADS=integer`
  → definisce quanti thread usare.
- `OMP_DYNAMIC=true/false`
  → abilita o meno l'adattamento dinamico del numero di thread.

Esempio (in bash):
export OMP_NUM_THREADS=8

export OMP_DYNAMIC=false

Lezione 4 - Decomposizione dei problemi e parallelismo multicore (MIMD-SM)

Quando si progetta un algoritmo **sequenziale**, non basta concentrarsi sul problema: bisogna considerare **chi lo esegue**(l'esecutore o l'architettura di calcolo).

Ogni software deve essere scomposto in **passi elementari** compatibili con il sistema (CPU singola o multicore). Per fare ciò ripartiamo dal modello numerico, analizzare gli N passi in modo da distribuirli eventualmente a più unità processanti.

Decomposizione Funzionale

Dividi il lavoro in compiti diversi

Idea centrale: Ogni unità processante esegue un passo differente del calcolo.

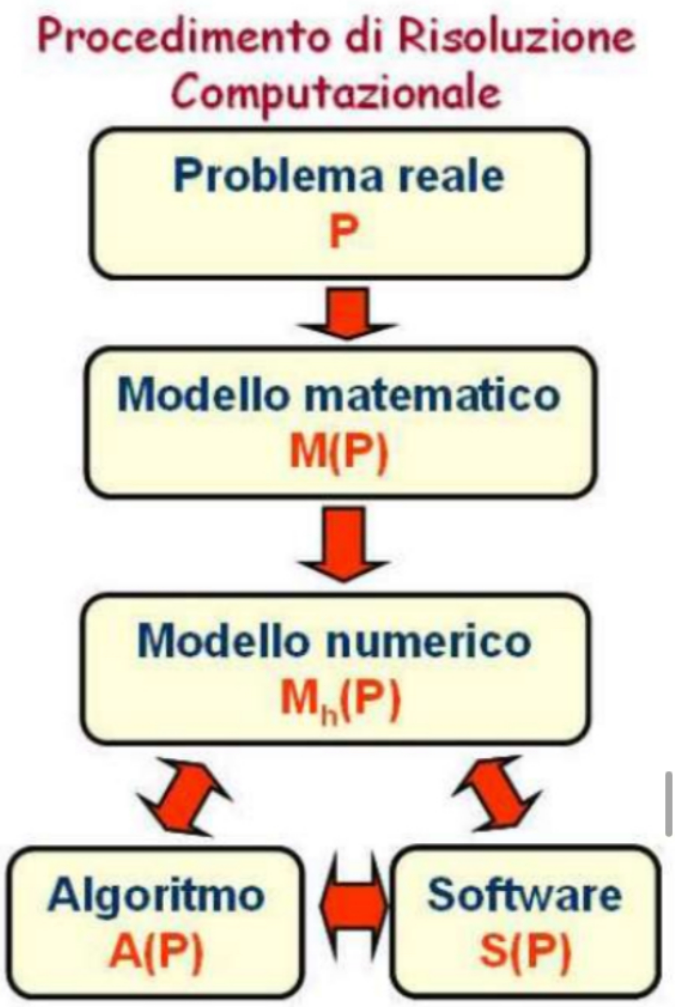
Il modello numerico viene decomposto in base a cosa deve essere fatto (le funzioni da eseguire).

Ogni processore fa qualcosa di diverso e indipendente.

Caratteristica fondamentale:
✓ Applicabile quando il modello ha più nuclei computazionali distinti
✓ Ogni nucleo rappresenta una fase o funzione del calcolo

Possiamo rappresentarlo come: $M_1(P), M_2(P), \ldots, M_N(P)$. Dove ogni $M_i(P)$ è un task o una unità funzionale distinta. Il modello è **decomposto in base al lavoro che deve essere svolto.**

Applicabile solo se il problema presenta più nuclei computazionali eterogenei, ovvero operazioni di natura diversa che possono procedere in parallelo.



Un **modello matematico** è la **traduzione in linguaggio matematico** di un fenomeno reale o di un problema concreto.Serve per **descrivere il comportamento del sistema** che vogliamo studiare, utilizzando equazioni, funzioni e variabili.

Un **modello numerico** è la **versione discretizzata e computabile** del modello matematico.
In altre parole, è ciò che otteniamo **quando trasformiamo le equazioni continue in un insieme finito di operazioni numeriche**, adatte all'elaborazione da parte di un calcolatore.

Una volta ottenuto il modello numerico $M_h(P)$, il problema diventa **computazionale**: dobbiamo eseguire gli N passi di calcolo.

E qui entra in gioco la lezione:

- ogni **passo h** o **insieme di dati** può essere distribuito su più **unità processanti** (core o CPU);
- a seconda di come lo si distribuisce, si parla di **decomposizione funzionale** o **decomposizione del dominio**.

Decomposizione del Dominio

Tutte le unità processanti eseguono **la stessa operazione** su **diversi sottoinsiemi di dati**.

Si tratta di dividere il *dominio dei dati* e processarlo contemporaneamente.

In questo caso il **lavoro è lo stesso**, ma su **parti distinte del compito**.
La fase finale (raffreddamento o unione) può richiedere **sincronizzazione**.

Possiamo rappresentarlo come: $M_{2.1}(P), M_{2.2}(P), \ldots, M_{2.np}(P)$. Dove np è il numero di unità processanti. Il modello è **decomposto in base al lavoro che deve essere svolto.**

Quindi in pratica, Abbiamo ad esempio il task, h=1 che è poi diviso in 1.1 e 1.2 o 1.np dove np sono le unità processanti

Vantaggiosa solo se la raccolta dei risultati locali è semplice o assente: cioè ogni core lavora su dati indipendenti e i risultati si combinano facilmente.

- Esempio serio Decomposizione del dominio con somma dei quadrati di un vettore:
  - Si divide il vettore in parti, e ognuna viene elaborata da un processore.
  - Ogni processore calcola il quadrato degli elementi della sua parte e produce i risultati localmente.
  - Infine, i risultati sono sommati insieme (fase di raccolta o riduzione).
- Decomposizione funzionale
  - Si dividono i passi in funzioni distinte e ogni processore fa un compito diverso.
  - P1 Calcola il quadrato
  - P2 Somma parziale dei quadrati calcolati da P1
  - P3 Somma finale
  - Quindi P1 elabora i dati, P2 combina i risultati, P3 somme finali.

Combinazione dei due approcci precedenti

Esiste anche una **combinazione** tra decomposizione funzionale e decomposizione del dominio, in cui:

- alcuni core eseguono funzioni diverse,
- altri elaborano sottoinsiemi di dati identici.

Esempio Somma di due vettori

Prendiamo come dati $a = (a_0, \ldots, a_{N-1}), b = (b_0, \ldots, b_{N-1})$ e ci aspettiamo un output pari a $c = (a_0 + b_0, \ldots, a_{N-1} + b_{N-1})$

Algoritmo sequenziale

Su un calcolatore monoprocessore la somma è calcolata eseguendo le N addizioni una per volta secondo un ordine prestabilito.

Quindi $i = 0...N-1 \rightarrow c_i = a_i + b_i$

Algoritmo Parallelo (MIMD-SM)

Se si dispone di np processori, si può **decomporre il problema**:

$$N \to \frac{N}{np}$$

Quindi andiamo a suddividere il problema e assegnamo la somma dei sottovettori ad ogni CPU.

Prendiamo come esempio un vettore di N=16 e abbiamo np = 4

Ogni core somma dei sottovettori:

- Core 1 = $a_0 + b_0, \ldots, a_3 + b_3$
- Core 2 = $a_4 + b_4, \ldots, a_7 + b_7$
- Core 3 = da 8 a 11
- Core 4 = da 12 a 15

I risultati parziali possono poi essere uniti in **memoria condivisa**.Tutti i core accedono contemporaneamente a **memoria globale su dati differenti**, calcolano le somme parziali e scrivono direttamente il risultato.

Questo è un problema **completamente parallelizzabile (*full parallel*)**, poiché non esiste dipendenza tra le operazioni. Con dipendenza si intende che due operazioni hanno bisogno del risultato di un'altra per potersi completare. In questo caso non avviene poiché 2 somme possono essere eseguite in contemporanea.

Esempio Somma di N numeri

Prendiamo come dati $a = (a_0, \ldots, a_{N-1})$ e vogliamo ottenere la somma totale dei componenti $sum = a_0 + \ldots + a_{N-1}$

Algoritmo Sequenziale

Su un calcolatore monoprocessore la somma è calcolata eseguendo le N-1 addizioni una per volta secondo un'ordine prestabilito.

$sumtot = a_0$ (Variabile)

Per $i = 1...N-1$ si esegue $somtot = somtot + a_i$ (In pratica un ciclo for come per i vettori)

Algoritmo Parallelo

Prendiamo anche qui N = 16 e np = 4

Dividiamo prima di tutto il vettore in np blocchi e andiamo ad assegnare ogni somma parziale ad una CPU, ottenendo cosi in questo caso 4 somme parziali:

- s_0
- s_1
- s_2
- s_3
- (L'ultimo elemento è s_{np-1})

Tutti i core accedono contemporaneamente a **memoria globale su dati differenti.**

Le somme parziale dovranno poi essere combinate per ottenere una somma totale:

$$sumtot = s_0 + s_1 + s_2 + s_3$$

Per fare questo ogni core C_i calcola la sua somma parziale S_i e deve addizionare tale valore ad una variabile come sumtot che conterrà la somma finale, MA i core devono accedere a sumtot uno alla volta. Per cui abbiamo 2 strategie di sincronizzazione degli accessi in memoria per fare ciò:

1 Strategia

Ogni core calcola la propria somma parziale(operazione concorrente)e ad ogni passo ciascun core aggiunge la propria somma parziale ad un unico valore (Sumtot). Il valore finale si trova nella memoria condivisa.

2 Strategia

Ogni core esegue una somma parziale (operazione concorrente) e ad ogni passo la metà dei core (rispetto al passo precedente) calcola un contributo della somma parziale(operazioni quasi concorrenti).

Diciamo che al passo 1 ogni ha fatto appunto le sue 4 somme parziali:

- s_0
- s_1
- s_2
- s_3

Al secondo passo la metà dei core (4:2 = 2) eseguono una ulteriore somma parziale fra ad esempio $s_0 + s_1$ e $s_2 + s_3$ poi avremo 1 solo core che sommerà i risultati precedenti portandoci al risultato finale. Quest'ultima operazione non è parallela poiché è solo 1 core ad eseguirla.

Lezione 5 - Intro Metriche

Efficienza di un algoritmo

L'efficienza si valuta in base a **tempi e risorse**.

Per un algoritmo sequenziale:

- **Complessità computazionale T(N)** → numero di operazioni eseguite.
- **Complessità spaziale S(N)** → numero di variabili utilizzate.

Nel caso di **calcolo parallelo**, però, il tempo non dipende solo dal numero di operazioni, ma anche da **quante operazioni vengono eseguite contemporaneamente**.

Prendiamo un paio di nomenclature prima vedendo la formula che abbiamo già visto:

$$\tau = k \cdot T(N) \cdot \mu$$

Il tempo di esecuzione di un software sequenziale è $\tau_1 = k \cdot T_1(N) \cdot \mu$

Dove:

- $T_1(N) =$ Complessità computazionale di un solo processore.
- $\mu = t_{calc} =$ tempo di esecuzione di un addizione floating point

---

Somma due vettori

Sequenziale

Facendo un esempio con la somma di vettori di dimensione N = 16 abbiamo 16 addizioni ovvero 16 passi temporali.

$$T_1(16) = 16, \text{ quindi } T(N) = N \text{ addizioni}$$

Quindi in un algoritmo sequenziale il numero complessivo di operazioni è uguale al numeri dei passi temporali.

Parallelo

Ipotizziamo di distribuire queste operazioni su 2 o 4 core.

Con **2 core**: ciascun core esegue 8 addizioni → 8 passi temporali
$\rightarrow T_2(16) = 8$

Con **4 core**: ciascun core esegue 4 addizioni → 4 passi temporali
$\rightarrow T_4(16) = 4$

Quindi come avevamo già visto in precedenza in generale $T_p(N) = \frac{N}{p}$.

Un calcolatore parallelo può eseguire più operazioni concorrentemente e il tempo di esecuzione non è proporzionale alla complessità di tempo, ovvero non dipende solo dal numero di operazioni floating point effettuate, questo porta a rendere la complessità di tempo non adatta a misurare l'efficienza di un algoritmo parallelo.

Come calcolare l'efficienza di un algoritmo parallelo

Quello che noi vogliamo fare è misurare quanto si riduce il tempo di esecuzione su p processori rispetto al tempo di esecuzione su 1 processore.

Come abbiamo visto prima in generale $T_p(N)$ viene calcolato come $T_p(N) = \frac{N}{p}$,

ma come facciamo a dire di quanto è più veloce dell'algoritmo sequenziale di base?

In questo ci aiuta lo Speed-UP.

Speed-Up

Lo speed-up misura la riduzione del tempo di esecuzione rispetto all'algoritmo su 1 core e si definisce con il rapporto :

$$S_p = \frac{T_1}{T_p}$$

Speed-up ideale: $S = p$

Speed-up reale: $S_p < p$

Maggiore è lo speed up, tanto più un algoritmo parallelo è veloce rispetto al corrispondente algoritmo sequenziale. Idealmente, ci aspettiamo che un algoritmo in parallelo su p processori sia p volte più veloce della corrispondente versione seriale, ma in realtà ogni algoritmo è composto da una parte seriale non parallelizzabile, che dunque restituirà un numero strettamente minore di p.Un algoritmo parallelo, quindi, è tanto più veloce quanto più lo speed up si avvicina allo speed ip ideale.

Vediamo nell'esempio precedente del calcolo della somme dei vettori:

p	T_p	$S = T_1/T_p$
1	16	1
2	8	2
4	4	4
8	2	8

Prendiamo ad esempio l'ultimo:

$T_1(16) = 16$ operazioni (questo sempre per i motivi visti prima)

$T_8(16) = 2$

Quindi $\frac{16}{2} = 8$

Come vediamo il tempo diminuisce esattamente p volte. Da questo posso capire finalmente la definizione generale "Con p core ci aspettiamo che T_1 sia p volte T_p"

Un algoritmo full parallel ha sempre speed-up ideale. Come nel caso della somma di due vettori.

Calcolo della somma di N = 16 numeri

I promossimi esempi non sono calcoli veri e propri, servono solo a mostrare come cambiano le grandezze di T_p per mostrare lo speed-up e nient'altro. Non sono quindi completi.

Vediamo ora un esempio un po più complesso. Questo come abbiamo visto in precedenza non è completamente parallalezzabile.

Algoritmo Sequenziale

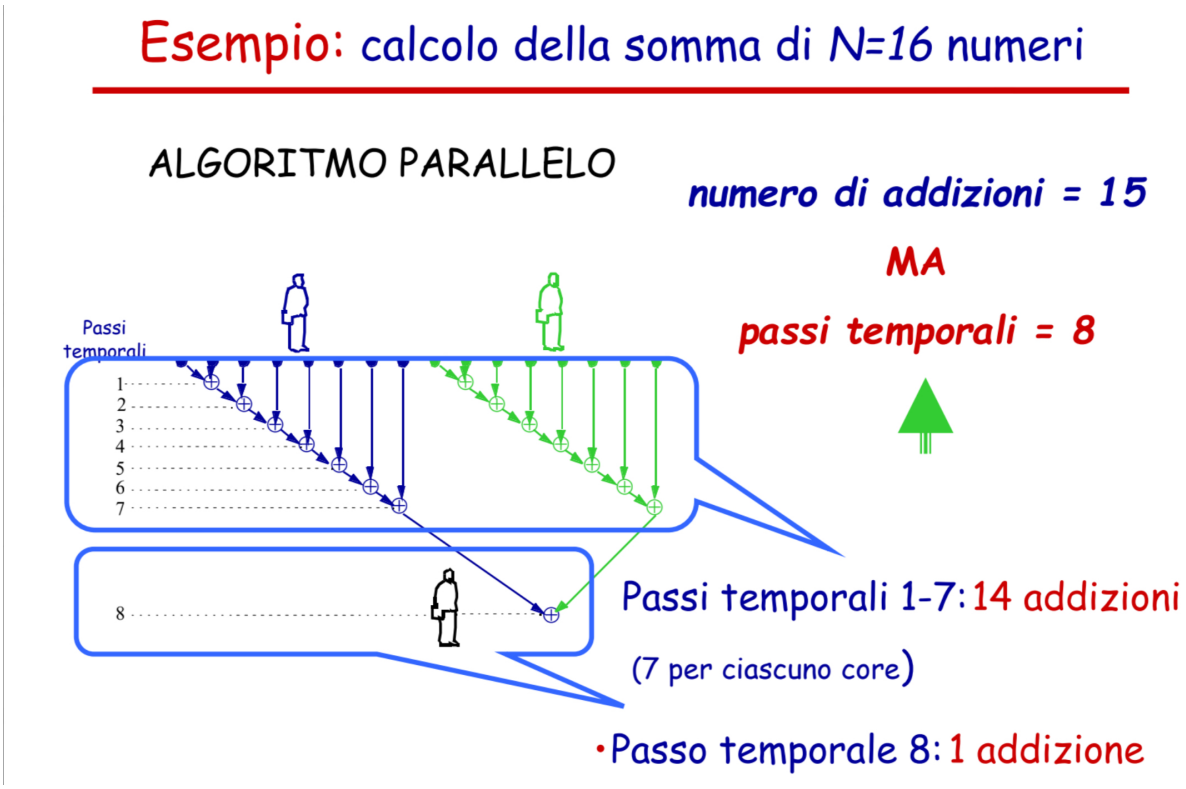
Di base nella somma di N numeri noi eseguiamo N-1 operazioni. Quindi il nostro $T_1(N)$= 15 e abbiamo 15 passi temporali.

$T_1(16) = (N-1) = 16-1 = 15$operazioni di addizione.Complessità tempo= N-1

Algoritmo Parallelo

Ipotizziamo adesso di usare p = 2 e usare la prima strategia di sincronizzazione:

Per adesso qui non viene data una formula perché verrà introdotta più avanti.



Ipotizziamo adesso di usare p = 2 e usare la <u>seconda strategia di sincronizzazione:</u>

Abbiamo anche qui 16 operazioni dove $T_1(16) = 15$, $T_2(16) = 8$ $T_4(16) = 5$ e $T_8(16) = 4$

<u>Il calcolo di $T_p(N)$ per la seconda strategia è pari a</u> $T_p(N) = \frac{N}{p} - 1 + log_2(p)$

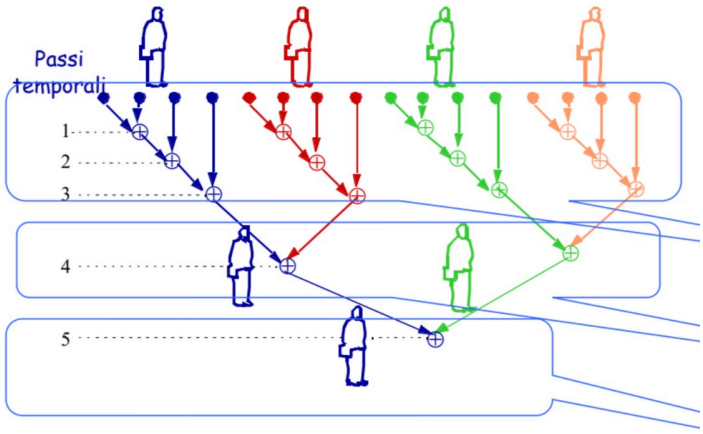
Questa formula tiene conto sia della **parte calcolabile in parallelo (N/p)** sia della **fase di riduzione ($log_2(P)$)** necessaria per combinare i risultati parziali.

$T_1 = 15$

$T_2 = 8 = (7 + 1)$

$T_4 = 5 = (3 + 2)$

$T_8 = 4 = (1 + 3)$



p	T_p	$S = T_1/T_p$
1	15	1
2	8	1.88
4	5	3.00
8	4	3,75

Possiamo notare come 8 sia il più veloce.

OverHead

L'Overhead totale misura quanto lo speed up differisce da quello ideale.

$$O_h = (pT_p - T_1)$$

Questa è una definizione che ci dice quanto tempo totale combinato è in più rispetto al tempo sequenziale.

Quello che noi vogliamo è isolare T_p cioè il tempo parallelo, per capire quanto pesa l'overhead nel calcolo del tempo effettivo.

$$O_h = p \cdot T_p - T_1 \rightarrow O_h + T_1 = pT_p \rightarrow T_p = \frac{O_h + T_1}{p}$$

Quindi spostiamo prima di tutto T_1 a sinistra per poi dividere entrambi i lati per p.

Andiamo poi a sostituire all'interno della definizione di speedup quello che abbiamo ottenuto:

$$S_p = \frac{T_1}{T_p} = \frac{T_1}{\frac{O_h + T_1}{p}} \cdot \frac{p}{p} = \frac{p \cdot T_1}{O_h + T_1} = \frac{p}{\frac{O_h}{T_1} + 1}$$

Qui ho sostituito per poi moltiplicare per p/p ed infine raggruppare T_1

Ora con questa formula abbiamo inserito l'overhead all'interno della formula per tenerne conto ottenendo lo speed up reale. E ci mostra che maggiore sarà O_h minore sarà lo speed-up.

Lo abbiamo fatto perché non siamo in un mondo ideale e quindi allo speed up dobbiamo dare anche conto dell'overhead per un calcolo preciso.

Dalla forma di overhead possiamo inoltre fare un'osservazione:
Riscrivendo la formula in funzione di p ed Oh, ci accorgiamo che al crescere di p, l'overhead aumenta.
Ciò significa che sebbene usare più processori comporta una riduzione del tempo di esecuzione, comporta anche maggior overhead, e dunque l'algoritmo risulta meno efficiente.

Esempio nell'algoritmo somma 2 strategia

$$T_1 = (N-1)$$

$$T_p = \frac{N}{p} - 1 + log_2(p)$$

$$O_h = pT_p - T_1 = [p(\frac{N}{p} - 1 + log_2(p)) - (N-1)] = (N - p + plog_2(p) - N + 1) = plog_2 p - p$$

Nelle slide troviamo una tabella con 2^k in pratica si generalizza una p che è potenza di 2.

Notiamo inoltre che al crescere di p l'overhead aumenta

Cosa significano in totale le due formule e come usarle

La forma normale dell Overhead ci indica quanto tempo extra si genera ad esempio:

$$T_1 = 15, T_p = 5, p = 4$$

$O_h = 5$ questo 5 sono i passi temporali in più rispetto al tempo ideale.

La seconda ci serve quando vogliamo capire quanto l'overhead riduce lo speed-up

A livello di risultato è identico fra la versione base della formula e quella con sostituito l'overhead entrambi calcolano lo speedup reale ma fare la sostituzione ci rende visibile come e quanto l'overhead influenza lo speed-up.

Se prendiamo ad esempio nella formula base il risultato ci dice quanto l'algoritmo è veloce ma non ci dice perché non arriviamo allo speed-up ideale.

Dove p è lo speed-up ideale e $1 + \frac{O_h}{T_1}$ è il fattore di penalizzazione. In pratica ci mette in evidenza l'impatto dell'overhead.

Efficienza

Lo speed-up non basta a fornire informazioni sull'efficienza dell'algoritmo, allora introduciamo il rapporto $E_p = \frac{S_p}{p}$. Questo valore più è grande e maggiore è lo sfruttamento dei processori. Quindi usare più processori non sempre è garanzia di sviluppo di algoritmi efficaci.

Se quel valore è uguale a 1 allora siamo in Efficienza Ideale.

Lezione 7 - Ware Amdhal

Ware Amdahl rappresenta una formula alternativa per calcolare lo speed up.

Abbiamo visto come calcolare lo speed-up e che misura la riduzione di complessità computazionale tra l'algoritmo sequenziale e parallelo. Ma quale algoritmo sclere per misurare T_1? Abbiamo 2 scelte:

- T_1 è la complessità dell'algoritmo parallelo per 1 unità processante
  - L'S_p di questa situazione ci da informazioni su quanto l'algoritmo si presta all'implementazione parallela.
  - Come svantaggio abbiamo era che l'algoritmo parallelo per 1 core potrebbe eseguire più operazioni del necessario.
- T_1 è la complessità dell'algoritmo parallelo sequenziale
  - L'S_p di questa implementazione ci da informazioni sulla riduzione effettiva della complessità nella risoluzione di un problema con p core.
  - La difficoltà è individuar il "Miglior" algoritmo sequenziale.

Per convenzione si usa la prima scelta e S_p ed E_p danno informazioni su quanto l'algoritmo si presta all'implementazione su architettura parallela.

Analisi Asintotica dello Speed-up

Per scrivere un buon algoritmo parallelo si deve partire da aver scritto un buon algoritmo sequenziale.

La complessità computazionale T_1 può essere suddivisa in 2 parti:

- T_s che è una parte relativa alle operazione che devono per forza essere eseguite in sequenziale
- T_c che è una parte relativa alle operazioni che potrebbero essere eseguite in parallelo

$$T_1 = T_s + T_c$$

Esempi

Somma di 2 vettori di dimensione n=4

Quindi il nostro $T_1(4) = 4, N = 4$ e p=2

Nella somma di due vettori non abbiamo nessuna parte sequenza ma abbiamo per 4 operazioni parallele, 2 eseguite da una CPU e 2 dall'altra.

$T_1(4) = T_s + T_c = 0 + (2 + 2) = 4$

Somma di N numeri n=4

Il nostro $T_1(4) = N - 1 = 4 - 1 = 3$ addizioni

Qui ci troviamo in una situazione in cui 2 addizioni potrebbero essere eseguite al passo 1 da 2 processori e 1 da un singolo, ovvero la finale.

$T_1(4) = T_s + T_c = 1 + 2 = 3$

Calcolo complessità parallela

$T_1 = T_s + T_c$ questa formula tiene conto di T_c come le operazioni che potrebbero essere parallele ma non lo sono ancora, perché stiamo tenendo ancora il caso a 1 processore, infatti è T_1. Andiamo quindi ora a tener conto che le operazioni della parte parallela sono eseguite concorrentemente da p core:

$$T_p = T_s + \frac{T_c}{p}$$

Riprendendo i 2 esempi di prima:

$T_2(4) = T_s + T_c = 0 + \frac{(2 + 2)}{2} = 2$, notiamo sempre 0 sequenziali e 2 parallele

$T_2(4) = T_s + T_c = 1 + \frac{2}{2} = 2$, qui 1 seq e 1 parallela per un totale di 2.

Quindi ricapitolando in generale, la complessità computazionale su p processori e non T_1 comprende i due componenti T_s e $\frac{T_c}{p}$.

Legge di Ware Amdahl

Adesso, prendiamo $T_s = a$ e $T_c = 1 - a$ e sostituiamo dentro $T_p = T_s + \frac{T_c}{p}$

ottenendo $T_p = a + \frac{1-a}{p}$ adesso prendiamo questo T_p e mettiamo nello speed-up:

$$S_p = \frac{T_1}{T_p} = \frac{1}{a + \frac{1-a}{p}}$$

Quello che abbiamo appena ottenuto è la legge di Amdahl.
È un modo alternativo per scrivere lo Speed-up che enfatizza la parte seq e la parte parallela.

Questa forma non si limita a dirci solo quanto l'algoritmo è più veloce ma ci dice perché non lo è di più. a fissa un limite superiore tecnico allo speed-up raggiungibile e non importa quanti processori aggiungiamo, la parte sequenziale resta il blocco.

Quindi questa nuova formula ci dice perché non posso guadagnare più veloci anche aumentando i core.

La versione con Overhead invece dava un limite allo speed up dovuto ai costi aggiuntivi di esecuzione parallela, quindi misura il degrado di prestazioni effettivo aumentando i core.

Prendiamo un piccolo esempio:

Diciamo che il 90% è parallelo quindi a = 0.1(sequenziale) il resto e il parallelo 0.9

P	Formula	Speed-Up Reale
1	(0.1+0.9/1) = 1	
2	(0.1+0.9/2) = 1.82	
4	(0.1+0.9/4) = 3.08	
8	(0.1+0.9/8) = 4.71	
16	(0.1+0.9/16) = 6.4	

infinito	(0.1+0) = 10	Limite massimo

Nell'ultimo è stato fatto il limite per infinito

Prendiamo ora i due esempi classici:

Somma di due vettori di N dimensione

Abbiamo a = 0 perché niente sequenziale e la parte 1-a = 1

$S_p = \frac{1}{\frac{1}{p}} = p$ che con p che tende ad infinito da infinito, significa che aumentando i

core lo speed up aum

enterà sempre.

Somma di N numeri

Sempre con p che tende ad infinito:

$$S_p = \frac{1}{a + \frac{1-a}{p}} = \frac{1}{a}$$

Vediamo adesso un paio di casi:

N Fissato e P variabile su somma N numeri

N = 32

P	A	$\frac{1-a}{p}$	S_p	E_p
2	0,032	0,968	1,9	0,95
4	0,032	0,903	3,4	0,85
8	0,032	0,775	5,1	0,6
16	0,032	0,516	6,2	0,3

Notiamo che la parte sequenziale è fissa per via dell'algoritmo e la parte parallela invece tende a degradare, infatti S_p e E_p man mano che la parte parallela diminuisce peggiorano.

Quel A = 0.032 si ottene formalmente tramite $\frac{T_s}{T_1} = \frac{1}{N-1} = \frac{1}{31} = 0.032$

Quindi se la dimensione di N è fissa e al crescere del numero di p le prestazioni peggiorano.

P fissato e N variabile

P = 2 e N che varia

N	A	1-a	$S_2(N)$	$E_2(N)$
8	0,14	0,86	1,75	0,875
16	0,06	0,94	1,8	0,9
32	0,03	0,97	1,9	0,96
64	0,01	0,99	1,96	0,99

Notiamo che la parte sequenziale tende a 0 la parte parallela e praticamente costante e vediamo che l'efficienza tende a quella ideale.

Quindi se p è fissato, al crescere della dimensione n del problema la parte sequenziale tende a 0.

Quindi per incrementare le performance la cosa migliore e bilanciare il numero di CPU e la dimensione del problema N. Ovviamente non possiamo incrementare N all'infinito dato che le risorse hardware sono limitate.

Da tutto questo abbiamo capito 2 cose:

- Fissato il size n del problema e aumento il numero p di CPU esiste p_{max} miglior numero di processori per risolvere il problema con l'algoritmo in esame e superando questo le prestazioni peggiorano.
- Fissato il numero p delle CPU e aumentando il size n del problema esiste n_{max} la massima dimensione che la macchina può memorizzare/elaborare e superando questo potrei avere problemi di memoria.

Lezione 8 - Isoefficienza

P e N Variabili

Somma di N Numeri

Applichiamo la legge di Ware Amdahl fissando il rapporto fra CPU e N pari a $\frac{n}{p} = 4$

n	p	a	$\frac{1-a}{p}$	$S_p(N)$	$E_p(N)$

8	2	0,14	0,86	1,75	0,875
16	4	0,06	0,8	3	0,75
32	8	0,03	0,77	5,1	0,64
64	16	0,01	0,76	9	0,56

Posiamo notare che la frazione delle operazioni eseguite in sequenziale tende a 0, quelle in parallelo è costante, lo speed up aumenta e l'efficienza e quasi costante.

Quindi aumento entrambi con **un rapporto fissato**, le prestazioni dell'algoritmo parallelo non degradano molto velocemente.

Fin ad ora ho legato questi 2 elementi secondo la regola n/p ma se questa regola non dovesse essere questa? In pratica quale dovrebbe essere il valore giusto per n/p?

Quello di cui noi abbiamo bisogno in partenza è che questo rapporto ci porti ad **un'efficienza costante**. Quindi bisogna trovare

$$I = \frac{n}{p}$$

Tale che l'efficienza resti appunto costante. Quello di cui ho bisogno è una funzione detta funzione di isoefficienza.

$$I = I(n_0, p_0, p_1)$$

Questa esprime la legge secondo cui si deve scegliere la nuova dimensione n_1 affinché si possa verificare che l'efficienza resti costante.

Calcolo Isoefficienza

L'isoefficienza per qualunque algoritmo è calcolata imponendo:

$$E_{p_0}(n_0) = E_{p_1}(n_1)$$

Però in realtà noi stiamo imponendo cosi:

$$E_{p_0}(n_0, p_0) = E_{p_1}(n_1, p_1)$$

Ricordiamo che l'efficienza è $E = \frac{S_p}{P}$ dove $S_p = \frac{p}{1 + \frac{O_h}{T_1}}$ con overhead, quindi:

$$E_{p_0}(n_0) = \frac{S_{p_0}(n_0)}{p_0} = E_{p_1}(n_1) = \frac{S_{p_1}(n_1)}{p_1}$$

Adesso andiamo a sostituire S_p:

$$\frac{S_{p_0}(n_0)}{p_0} = \frac{p_0}{p_0(\frac{O_h}{T_1(n_0)} + 1)} = \frac{1}{\frac{O_h}{T_1(n_0)} + 1}$$

$$\frac{S_{p_1}(n_1)}{p_1} = \frac{p_1}{p_1(\frac{O_h}{T_1(n_1)} + 1)} = \frac{1}{\frac{O_h}{T_1(n_1)} + 1}$$

Otterremo:

$$\frac{1}{\frac{O_h}{T_1(n_0)} + 1} = \frac{1}{\frac{O_h}{T_1(n_1)} + 1}$$

qui possiamo usare $\frac{1}{A} = \frac{1}{B}$ che ci permette di riscrivere come: $A = B$ e da qui ho già sostituito O_h con le versioni appropriate come detto prima:

$1 + \frac{O_h(n_0, p_0)}{T_1(n_0)} = 1 + \frac{O_h(n_1, p_1)}{T_1(n_1)}$ semplifichiamo l'1:

$\frac{O_h(n_0, p_0)}{T_1(n_0)} = \frac{O_h(n_1, p_1)}{T_1(n_1)}$ si porta poi T_1 al numeratore(il valore non cambia):

$\frac{T_1(n_0)}{O_h(n_0, p_0)} = \frac{T_1(n_1)}{O_h(n_1, p_1)}$ qui abbiamo una classica proporzione del tipo: $\frac{A}{B} = \frac{C}{D}$ che possiamo riscrivere come $A : B = C : D \rightarrow A = \frac{B}{D} * C$ e quindi arrivo a:

$$T_1(n_1) = \frac{O_h(n_1, p_1)}{O_h(n_0, p_0)} T_1(n_0)$$

Calcolo Isoefficienza nella somma 2 strategia

Sappiamo che l'overhead è $O_h(n, p) = plog_2 p - p$

In sequenziale è pari a $T_1(N) = N - 1$

Andiamo a sostituire nella formula:

$$n_1 - 1 = \frac{p_1 log_2 p_1}{p_0 log_2 p_0}(n_0 - 1)$$

Nel passare da p_0 a p_1 con $p_1 > p_0$, la dimensione del problema deve aumentare, rispetto alla dimensione iniziale n_0, del fattore:

$$I = \frac{p_1 log_2 p_1 - p_1}{p_0 log_2 p_0 - p_0}$$

Detto ciò un'algoritmo prende il nome di **scalabile** se **l'efficienza rimane costante al crescere del numeri dei processori** e della dimensione del problema, in un rapporto costante pari a $\frac{O_h(n_1, p_1)}{O_h(n_0, p_0)}$.

Ora controlliamo se la somma è un algoritmo scalabile:

Prendiamo una base di partenza di $p_0 = 4$,$n_0 = 64$ vogliamo adesso calcolare n_1 per $p_1 = 8$ Quindi passare a 8 processori e vogliamo vedere quale n ci serve per rimanere efficienti.

$$I = \frac{p_1 log_2 p_1 - p_1}{p_0 log_2 p_0 - p_0} = \frac{8 \cdot 3 - 8}{4 \cdot 2 - 4} = \frac{16}{4} = 4$$

Quindi il nostro $n_1 = n_0 \cdot 4 = 256$

Se facessimo la stessa cosa con $p_2 = 16$ da sostituire a p_1 nella formula otterremo 12 che ci porterà a 768.

Questo significa che se quadruplico i processori da 4 a 16 devo aumentare il problema di 12 volte per avere una efficienza costante.

Calcoliamo adesso i nostri $T_p(N) = (\frac{n}{p} - 1 + log_2(p))$, dove questo ricordo è la formula per la seconda strategia.

Avremo per 64,256 e 768 i valori pari a

$T_4(64) = 17$

$T_8(256) = 34$

$T_16(64) = 51$

Calcoliamo ora i vari speed-up:

$$\frac{63}{17} = 3.71$$

$$\frac{255}{34} = 7.5$$

$$\frac{767}{51} = 15.04$$

Ed infine calcoliamo le loro Efficienze:

	1	4	8	16
64	1.0	0.93		
256	1.0		0.93	
768	1,0			0.93

L'efficienza rimane come vediamo costante al crescere di p e di n con la legge di isoefficienza. La somma è quindi scalabile.

Utilizzo delle metriche nei 2 algoritmi

Nel calcolo dello speed up viene usata la forma normale perché ci permette di calcolare lo speed-up, efficienza e overhead reali di ogni algoritmo invece la versione in funzione dell'overhead ci permette di studiare come l'overhead influenza la scalabilità e derivare la legge di isoefficienza. Quindi la formula con overhead ci dice da cosa dipende lo speed up invece la classica ti dice quanto vale davvero lo speed up.

Algoritmo somma di vettori

p = 8 e i due array di dimensione N

Ogni core fa $\frac{N}{p}$ somme indipendenti (parziali) senza nessuna somma finale

Abbiamo un tempo sequenziale pari a $T_1(N) = N$ e un parallelo $T_p(N) = \frac{N}{p}$

poiché ogni core lavora contemporaneamente sul proprio blocco.

Lo speed-up $S_p = \frac{T_1}{T_p} = \frac{N}{\frac{N}{p}} = p$ abbiamo quindi lo speed-up ideale.

Efficienza $E_p = \frac{S_p}{p} = \frac{p}{p} = 1$ ideale anche questo e infine l'overhead

$O_h = pT_p - T_1 = p(\frac{N}{p}) - N = 0$ che non è presente.

Caso non divisibile per 2

Se N non è divisibile per p il lavoro non si divide equamente e alcuni processori faranno una somma in più. Di conseguenza il tempo $T_p(N) = int(\frac{N}{p}) + 1$ cioè tutti i processori eseguiranno un'operazione in più per compensare. Con int intendiamo la parte intera di quella divisione.

Algoritmo somma di N numeri 1 Strategia

Ipotizziamo p = 8 e N numeri

Ogni core esegue le sue $\frac{N}{p} - 1$ somme parziali

Questo algoritmo ora dopo aver fatto le somme parziale deve fare la somma totale che è sequenziale ed esegue $p - 1$ somme sequenziali.

Il tempo sequenziale per l'intero algoritmo è $T_1(N) = N - 1$ invece per quando riguarda $T_p(N) = (\frac{N}{p} - 1) + (p - 1)$

Calcoliamo ora lo speedUP $S_p = \frac{T_1}{T_p} = \frac{N-1}{(\frac{N}{p} - 1) + (p - 1)}$ che sarà < p

Con questo < p questo perché abbiamo una parte sequenziale che non può essere distribuita e quindi non raggiungeremo lo speed-up ideale e sarà sempre minore di p. E quando il tempo parallelo sarà molto piccolo la parte sequenziale sarà effettivamente dominante nel tempo totale.

Vediamo l'efficienza : $E_p = \frac{S_p}{p} = \frac{N-1}{p[(\frac{N}{p} - 1) + (p - 1)]}$ che nel nostro caso ipotetico N non cambia e sappiamo che al crescere di p l'efficienza sarà peggiore. Inoltre questo è dimostrabile anche risolvendo il denominatore che ci darà una

crescita quadratica $N + p^2 - 2p$ e più grande è p, più grande sarà il denominatore e di conseguenza minore sarà E_p.

$$O_h = pT_p - T_1 = p[(\frac{N}{p} - 1) + (p - 1)] - (N - 1) = p^2 - 2p + 1$$

La crescita asintotica è quadratica, quindi non è scalabile.

Caso non divisibile per 2

Anche qui nel caso non sia divisibile $T_p(N) = int(\frac{N}{p}) + (p - 1)$. Quindi ogni processore una somma in più.

Quantità	**Formula**	**Effetto quando cresce p**
(T_p)	((N/p - 1) + (p - 1))	diminuisce poco perché cresce la parte seriale
(S_p)	((N-1)/T_p)	rallenta presto
(E_p)	(S_p/p)	scende al crescere di p
(O_h)	(pT_p - T_1)	cresce molto con p

Algoritmo somma di N numeri 1 Strategia

Qui abbiamo lo stesso obiettivo di prima sempre p=8 e N

Ogni core esegue le sue $\frac{N}{p} - 1$ somme parziali

Ma adesso nella fase 2 abbiamo la somma totale ad albero che ha un tempo di $log_2(p)$ che sono poi i livello di somma.

Il tempo sequenziale $T_1(N) = N - 1$ sempre algoritmo ad 1 processore.

$$T_p(N) = (\frac{N}{p} - 1) + log_2(p)$$

Lo speed up in questo caso $S_p = \frac{T_1}{T_p} = \frac{N - 1}{(\frac{N}{p} - 1) + log_2(p)}$

In questa strategia la riduzione non è completamente parallela ma neanche sequenziale nel senso vero della parola. Ma è semi sequenziale nel senso che non viene ne diviso per i processori e in più non possiamo calcolare il livello 2 senza

aver finito il primo livello. Quindi non è perfettamente parallelizzabile, quindi è seriale per livello cioè richiede passi successivi che non si possono comprimere ulteriormente.

Questa parte vediamo che è un logaritmo di conseguenza cresce molto lentamente e questo porta l'algoritmo ad avvicinarsi di più allo speed-up ideale.

$E_p = \frac{S_p}{p} = \frac{N-1}{p[(\frac{N}{p}-1)+log_2(p)]}$ stesso discorso vale per efficienza che per via del logaritmo scende molto meno.

$$O_h = pT_p - T_1 = p[(\frac{N}{p}-1)+log_2(p)] - (N-1) = plog_2(p) - p + 1$$

Anche l'overhead cresce in modo quasi lineare che è molto molto meno del quadrato della prima strategia.

Caso non divisibile per 2

Anche qui nel caso non sia divisibile $T_p(N) = int(\frac{N}{p}) + log_2(p)$. Quindi ogni processore una somma in più.

Lezione 11 - Utilizzo di tutto sui 2 algoritmi

Piccolo riassunto delle metriche:

- **Speed-UP**: misura di quanto si riduce il tempo di esecuzione passando da 1 processore a p processori: $S_p = \frac{T_1}{T_p}$
- **Overhead**: L'overhead totale misura quanto in "in più" serve rispetto al caso ideale, è la differenza tra il tempo totale impiegato da tutti i processori pT_p e il tempo che servirebbe con un solo T_1. Da questa definizione si ricava sostituendo nella formula dello speed up una forma nuova che ci mostra direttamente che più overhead abbiamo, più lo speed-up diminuisce.
- **Efficienza**: Misura quanto bene i processori vengono realmente utilizzati
- **Ware-Amdahl:** La formula mostra il limite teorica dello speed-up: anche con infiniti processori, la parte sequenziale impone un limite massimo.
- **Isoefficienza**: La scalabilità indica la capacità di mantenere le prestazioni aumentando sia p che N e si mantiene costante un rapporto chiamato

isoefficienza, che definisce come deve crescere N rispetto a p per mantenere l'efficienza invariata.

Fra ware Amdahl e La versione base la differenza principale è che nonostante diano lo stesso risultato finale, la versione base è utile quando conosciamo i tempi. L'altra è utile se vogliamo capire come lo speed-up cambia in base a quanta parte del codice si può parallelizzare.

Adesso riprendiamo un attimo Ware-Amdahl

$$S_p = \frac{1}{a + \frac{1-a}{p}}$$

- a = frazione sequenziale
- (1-a) = frazione parallelizzabile
- $\frac{1-a}{p}$ = frazione eseguita realmente in parallelo

Questa forma si usa solo se possiamo distinguere con chiarezza quali istruzioni sono parallele e quali sono sequenziali.

Amdahl Generalizzato

Vediamo adesso una versione generalizzata:

$$S_p = \frac{1}{a_1 + \sum_{k=2}^{p-1} \frac{A_k}{k} + \frac{a_p}{p}}$$

- a_1 frazione di operazioni eseguite da un solo processore
- a_k frazione di operazioni eseguite con parallelismo medio $k < p$
- a_p frazione di operazioni eseguite con parallelismo totale p

Proviamo adesso ad usare entrambe le versioni sui 2 algoritmi già visti:

Somma di due vettori di dimensione N

Prendiamo come riferimento p = 8 e la dimensione dei due vettori di 32.

In sequenziale eseguiamo N somme quindi 32.

Ogni processore eseguirà un somma parziale pari a 4 somme ciascuno quindi:

$nloc = \frac{N}{p} = \frac{32}{8} = 4$ somme parziali ad ogni core. $4 \cdot 8 = 32$ sono totali eseguite, ripartire in ogni core.

Andiamo a dimostrare che il nostro $1 - a = 1$ ovvero che non abbiamo operazioni sequenziali. Questo lo facciamo cosi: $1 - a = \frac{operazioniparallele}{operazionitotali} = \frac{32}{32} = 1$ da cui vediamo che a = 0.

Da questo possiamo vedere che la parte parallela è pari a $\frac{1-a}{p} = \frac{1}{\frac{1}{8} + 0} = \frac{1}{8}$

Calcolando ora lo speed up otteniamo che $S_p = \frac{1}{0 + \frac{1}{p}} = \frac{1}{0 + \frac{1}{8}} = 8$

Da questi risultati vediamo che è un angoliamo FULL-PARALLEL.

Somma di N numeri 1 strategia

Abbiamo N = 32, P=8 e Ogni core elabora nloc = N/p = 4 numeri

Vediamo la prima fase:

In sequenziale abbiamo N-1 somme, cioè 31.

Ora ogni core eseguire N-1 somme parziali quindi $nloc - 1 = 4 - 1 = 3$

Adesso $3 \cdot 8 = 24$ sono le somme eseguite in parallelo delle 31 totali.

La nostra frazione parallelizzabile $1 - a = \frac{24}{31}$

Andiamo a calcolarci ora la nostra frazione parallela su più core:

$$\frac{1-a}{p} = \frac{\frac{24}{31}}{8} = \frac{24}{31} \cdot \frac{1}{8} = 3 \cdot \frac{1}{31}$$

Ora andiamo passiamo alla seconda fase dell'algoritmo:

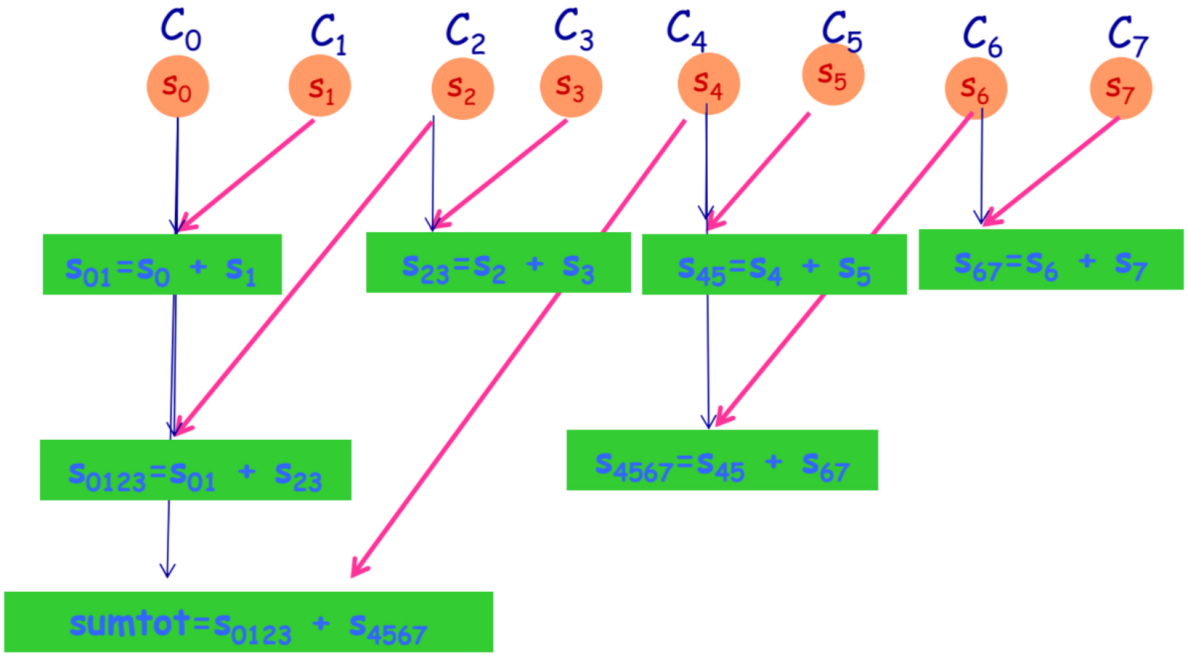
Bisogna adesso aggiornare la somma totale che viene fatto in p-1 passi ovvero $p - 1 = 8 - 1 = 7$ somme

Come notiamo sono le 7 rimanenti ovvero le parziali. Di conseguenza la nostra parte parziale è pari a $\frac{7}{31}$

Andiamo ora a calcolare lo speed up: $S_p = \frac{1}{\frac{7}{31} + \frac{3}{31}} = \frac{31}{10} = 3{,}1.$

N numeri Seconda Strategia

Ora però con la seconda strategia non riusciamo a distinguere bene la parte parallela e la parte sequenziale.



Per quanto riguarda la fase 1 abbiamo una situazione identica alla prima strategia.

Quindi con 8 core abbiamo esattamente $\frac{24}{31}$ somme parallelizzabili quindi

$\frac{1-a}{p} = \frac{\frac{24}{31}}{8} = \frac{24}{31} \cdot \frac{1}{8} = 3 \cdot \frac{1}{31}$ parallele.

Ricordiamo adesso il tipo di strategia dove dimezziamo i core attivi ad ogni nuova fase. Quindi dobbiamo andare a calcolarci adesso per ogni fase le sue somme parziali.

Nella prima abbiamo visto che tutti i core lavorano in parallelo, ora nella seconda con 4 core abbiamo 1 somma fatta contemporaneamente da 4 core.

1 è la somma parziale e quindi $1 * 4 = 4$ delle 31 somme viene fatta in questa fase.

La parte parallela è $\frac{a_4}{31} = \frac{1}{31}$

Eseguiamo sempre 1 parziale su 2 core e 2 somme delle 31 parallelizzabili con parte parallela $\frac{a_2}{31} = \frac{1}{31}$.

Ultima fase sempre 1 somma, che sarà 1 sulle 31, parte parallela:

$$\frac{a_1}{31} = \frac{1}{31}$$

Da tutte questo notiamo che per 8,4,2 e 1 vengono eseguite delle somme invece per il resto come la fase con 7,6,5,3 core non esiste proprio, quindi vale 0.

Andiamo ad usare la generalizzata:

$$S_p = \frac{1}{\frac{1}{31} + \frac{1}{31} + \frac{1}{31} + \frac{3}{31}} = \frac{31}{6} = 5{,}1$$

Abbiamo usare la generalizzata perché il parallelismo varia di fase in fase.

N non divisibile per 2

Somma Vettori

Diciamo p = 8 e N = 67

Come abbiamo sempre visto prendiamo la parte intera che è come dire “come tutti i core avessero 9 elementi”.

$$T_p(N) = (\frac{N}{p} + 1) \cdot t_{calc}$$

t_{calc} è una singola operazione ed è messo li per rendere generale il modello temporale. Ad esempio nel nostro caso sarà $9t_{calc}$

Nel conto delle frazioni, qui la fase è **puramente parallela**: la parte utile è tutta parallela e si distribuisce come $a_8 = \frac{3*9+5*8}{67} = 1$

Dunque il nostro $(1-a) = \frac{67}{67} = 1$ e a=0

Abbiamo quindi 3 come numeri in più a quelli divisibili per p.

Somma N 1 strategia

Anche qui andremo a prendere la parte intera della divisione come anche visto precedentemente con t_{calc} alla fine.

Ricordiamoci però qui che le somme sono sempre N-1. Quindi

Avremo $a_8 = \frac{3*8+5*7}{66}$ quindi 5 core eseguiranno 7 somme e altri 3 8 per un totale di 66.

Possiamo poi usare ware-amdahl per $S_p = \frac{1}{\frac{7}{66} + \frac{\frac{59}{66}}{8}} = 4{,}59$

$E_p = 0{,}57$

Somma N 2 Strategia (Da solo)

Abbiamo 8 core e 59 somme su 66 in a_8. Abbiamo poi come prima le riduzioni man mano:

$a_4 = \frac{4}{66} \qquad a_2 = \frac{2}{66} \qquad a_1 = \frac{1}{66}$

Andando ad usare la generalizzata otteniamo 6,36 e come efficienza 0,80

Isoefficienza

Viene ridata nuovamente la definizione:

Presa da CHATGPT: fissati p0 e n0, si sceglie n1 in modo che il rapporto degli overhead rimanga costante garantendo cosi stessa efficienza al crescere di p. In pratica per l'algoritmo si calcola O_h(p,n) poi si impone un rapporto costante e si ricava n1.

2 Vettori

$$T_1(N) = N$$

$$T_p(N) = \frac{N}{p}$$

In questo caso otteniamo nelle slide una forma indeterminata 0/0.

Per gli algoritmi full parallel l'isoefficienza può assumere qualunque valore poiché naturalmente scalabili.

N 1 strategia

$$T_1(N) = N - 1$$

$$T_p(N) = (\frac{N}{p} - 1) + (p - 1)$$

Otteniamo:

$$O_h = p(\frac{N}{p} - 1 + p - 1) - (N - 1) = p^2 - 2p + 1 = (p - 1)^2$$

$$I = \frac{p_1^2 - 2p_1 + 1}{p_0^2 - 2p_0 + 1}$$

l'overhead cresce **quadraticamente** con p; per mantenere la stessa efficienza, N deve crescere **almeno** in modo da compensare $(p - 1)^2$.

N 2 strategia

$$I = \frac{p_1 log_2(p)_1 - p_1}{p_0 log_2(p)_0 - p_0}$$

l'overhead cresce **quadraticamente** con p; per mantenere la stessa efficienza, N deve crescere **almeno** in modo da compensare $(p - 1)^2$.

Altre Esercitazioni Su Slide Non fatte

Lezione 14 - Prodotto Scalare

Algoritmo

Definiamo cos'è il prodotto scalare:

$a = (a_0, a_1, \ldots, a_{N-1})$

$b = (b_0, b_1, \ldots, b_{N-1})$

Con output pari a $c = (a_0 b_0 + a_1 b_1 + , \ldots, + a_{N-1} b_{N-1})$

Algoritmo sequenziale

Su un calcolatore monoprocessore il prodotto scalare è calcolato eseguendo:

- N moltiplicazioni una per volta
- N-1 addizioni una per volta

———————————————-

c := 0

for i = 0 to N−1 do

  c := c + (a[i] × b[i])

endfor

———————————————-

Algoritmo parallelo

L'algoritmo funziona in 2 fasi, abbiamo la prima fase dove ogni processore esegue una somma parziale, per dirlo meglio ad esempio un processore eseguirà da 1 a 3 moltiplicazioni es:(a_1*b_1,…) per poi sommare i componenti del proprio sottovettore, portando cosi una somma parziale. Dopodiché la collezione avviene tramite le 2 strategie che abbiamo sempre visto.

Calcolo Speed-Up, OverHead ed Efficienza

Il nostro $T_1 = 2N - 1$ questo perché adesso 1 processore in sequenziale dovrà eseguire non solo gli N prodotti ma anche le N-1 somme.

Ora dobbiamo vedere la divisione per i vari processori:

$\frac{N}{p}$ prodotti e $\frac{N}{p} - 1$ somme.

Portandoci ad un $T_p(N) = 2\frac{N}{p} - 1$ a cui aggiungeremo la raccolta dei dati

A questo punto dobbiamo scegliere come collezionare i dati

1 strategia

Ipotizziamo sempre un p=8

$$T_p(N) = 2\frac{N}{p} - 1 + (p-1)$$

$$S_p = \frac{T_1(N)}{T_p(N)} = \frac{2N-1}{2\frac{N}{p} - 1 + (p-1)}$$

$$O_h = pT_p(N) - T_1(N) = p[2\frac{N}{p} - 1 + (p-1)] - (2N-1)$$

$$E_p(N) = \frac{S_p}{p} = \frac{2N-1}{p[2\frac{N}{p} - 1 + (p-1)]}$$

2 strategia

$$S_p = \frac{T_1(N)}{T_p(N)} = \frac{2N-1}{2\frac{N}{p} - 1 + log_2(p)}$$

$$O_h = pT_p(N) - T_1(N) = p[2\frac{N}{p} - 1 + log_2(p)] - (2N-1)$$

$$E_p(N) = \frac{S_p}{p} = \frac{2N-1}{p[2\frac{N}{p} - 1 + log_2(p)]}$$

Se N non divisibile per p

INDOVINA??? Anche qua prendiamo la parte intera +1 cosi da dire che tutti i processori debbano fare esempio 9 prodotti puntuale e poi passando alle somme stesso concetto prendiamo l'interno di N/p cosi da avere poi un totale:

$$int(N/p) + 1 + int(N/p) = 2int(N/p) + 1$$

La parte di collezione rimane invariata ma bisogna tenere conto dell'1 all'interno del tempo parallelo $T_p(N)$

$$T_p(N) = 2int(\frac{N}{p} + 1 + (p-1)$$

$$T_p(N) = 2int(\frac{N}{p} + 1 + log_2(p)$$

Isoefficienza

1 Strategia

$$O_h = p[2\frac{N}{p} - 1 + (p-1)] - (2N-1) = 2p\frac{N}{p} - p + p^2 - p - 2N + 1 = 2N + p^2 - 2N + 1 - 2p = p^2 - 2p + 1$$

$$I = \frac{p_1^2 - 2p_1 + 1}{p_0^2 - 2p_0 + 1}$$

2 Strategia

$$O_h = p[2\frac{N}{p} - 1 + log_2(p)] - (2N-1) = 2p\frac{N}{p} - p + plog_2(p) - 2N + 1 = 2N - p + plog_2(p) - 2N + 1 = plog_2(p) - p + 1$$

$$I = \frac{p_1 log_2(p_1) - p_1 + 1}{p_0 log_2(p_0) - p_0 + 1}$$

Ware-Amdahl generalizzata

Prendiamo come esempio p=8 e i 2 vettori pari a N=32 con nloc = 4

In sequenziale le operazioni totali che faremmo sono : 2N-1 = 63 operazioni.

La prima fase come abbiamo detto già prima è composta da nloc + nloc-1.

Ovvero 4 prodotti e 3 somme per un totale di 7 operazioni.

Adesso 7 * 8 = 56 delle 63 operazioni totali.

Quindi $a_8 = \frac{56}{63}$

Adesso dobbiamo alle 2 strategie di somme parziali

Abbiamo quindi 1-a = 7/63

1 Strategia della somma applicata

Tutte le somme vengono eseguite da 1 solo core. Quindi tutte le fasi da 7 a 2 sono 0.

Quindi $a_1 = \frac{7 * 1}{63}$ e $a_8 = \frac{56}{63}$

2 Strategia della somma applicata

Abbiamo le somme distribuite come sempre.

Nella prima fase è sempre uguale poi però dobbiamo calcolare le fasi restanti:

$$a_4 = \frac{4 * 1}{63}$$

$$a_2 = \frac{2 * 1}{63}$$

$$a_1 = \frac{1 * 1}{63}$$

$$a_8 = \frac{56}{63}$$

Cosa succede se N non divisibile per p

Quello che cambia in entrambe le strategia è a_8

NLOC è il numero di Elementi assegnato a ciascun core come base. R è il resto della divisione tra N e P.

Prendiamo il caso N=54, p=8, nloc=6, r=6

In sequenziale = $2N - 1 = 107$

6 core elaborano 13 operazioni e 2 ne fanno 11:

$$a_8 = \frac{13 * 6 + 11 * 2}{107} = \frac{100}{107}$$

Lezione 17 - Convoluzione Monodimensionale

La convoluzione è un'operazione tra due funzioni di una variabile che consiste nell'integrare il prodotto tra la prima e la seconda traslata di un certo valore.

Il risultato della convoluzione viene solitamente indicato con: $h = f \cdot g$

Un’ottimo utilizzo della convoluzione potrebbe essere ad esempio un filtro di sfocatura delle immagini. Dove A = valori di luminosità di un immagine. C è il filtro simmetrico smussante e B immagine risultante più morbida. Ogni pixel di B sarà una media dei pixel vicini di A permettendo di ridurre variazioni brusche.

Definizione del problema

Si considerano due funzioni $f(x)$ e $g(x)$ definite come $f : \mathbb{R} \to \mathbb{R}$ ovvero da R in sé.

Si definisce convoluzione di f e g la funzione definita come:

$$(f \cdot g)(x) = \int_{-\infty}^{+\infty} f(\tau)g(x-\tau)d\tau$$

- Consideriamo x come una variabile indipendente che rappresenta il punto specifico nel quale vogliamo valutare il risultato della convoluzione delle due funzioni.

$f : \mathbb{R} \to \mathbb{R}$ (La funzione prende un numero reale come ingresso e ne restituisce uno reale come uscita)

$\tau \to f(\tau)$ (significa che la funzione f prende un valore di input rappresentato da τ e restituisce un valore rappresentato da $f(\tau)$, può essere detto anche come x è mappato come f(x)) che spesso rappresenta un segnale o una risposta di impulso. Nel dominio della convoluzione consideriamo $f(\tau)$ come segnale di input. Invece $g(\tau)$ che è un’altra funzione di τ, spesso interpretata come la funzione di risposta di un sistema o un filtro applicato a f.

Definizione nel contesto continuo

$$(f \cdot g)(x) = \int_{-\infty}^{+\infty} f(\tau)g(x-\tau)d\tau$$

- $f(\tau)$: è il segnale di ingresso, valutato sulla variabile di integrazione τ
- $g(x-\tau)$: la funzione g è rovesciata e traslate di x
  - Rovesciata (o meglio riflessa dal lato opposto, invece che verso destra va verso sinistra) perché l’argomento è $-\tau$
  - Traslata (spostata di una quantità x) perché si aggiunge x

- Questo è importante farlo perché in pratica g “scorre” sopra f, e in ogni posizione x calcoliamo quanto le due funzioni coincidono (moltiplicandole e sommando i prodotti)

- $f(\tau)g(x-\tau)$ rappresenta il contributo di f e g in quel punto
- L’integrale somma tutti i questi contributi per ottenere il valore complessivo in x.
- τ è una variabile di integrazione che scorre nel tempo per indicare la traslazione di una delle funzioni.

Con contesto continuo si intende l’utilizzo di una funzione continua ovvero definita su infiniti punti, cioè il suo dominio non ha buchi come ad esempio f(x) = sin(x).

[L’integrale si usa perché abbiamo infiniti punti da sommare]

Definizione nel contesto finito

[Si usa la sommatoria perché sono pochi valori]

La definizione è questa e una sequenza finita è invece una lista di valori separati e in numero limitato come ad esempio $f = [3,5,2,1]$

$$(f \cdot g)[n] \sum_{m=0}^{min(M-1,n)} f[m] \cdot g[n-m] \qquad n \in [0, M+N-2]$$

Nel momento in cui f e g sono finiti:

- f[m] e g[n] sono i valori delle due sequenze.
- M: lunghezza di f
- N: lunghezza di g
- L’intervallo $n \in [0, M+N-2]$ invece è la lunghezza della sequenza coinvolta, cioè lunghezza massima del risultato
- Il termine g[n-m] indica che stiamo rovesciando e traslando g esattamente come nel caso continuo.

Esempio Numerico caso finito

$f = [3,5,2,1], g = [1,2]$

Calcoliamo $(f * g)[2]$

$$(f \cdot g)[n] \sum_{m=0}^{2} f[m] \cdot g[2-m] = f[0] * g[2] + f[1] * g[1] + f[2] * g[0] = 0 + 5 * 2 + 2 * 1 = 12$$

Convoluzione Discreta Monodimensionale

Prendiamo come input: array A di lunghezza N e array C di raggio k, lunghezza 2k+1

Ci aspettiamo un output: Array B di lunghezza N.

Con raggio K intendiamo un filtro, cioè prendiamo in considerazione alcuni elementi vicino a quello centrale di A. Il numero k indica quanti elementi a destra e a sinistra dal punto centrale consideriamo valori. Per questo prende il nome di raggio del filtro.

Se il filtro ha raggio k allora: k a sinistra dal centro, k a destra dal centro + 1 elemento centrale = 2k+1. Piccolo esempio: Se k = 2 allora avremo 5 elementi tipo una cosa del genere (-2 -1 0 +1 +2)

Quando calcoleremo B[i] siamo dicendo in pratica di calcolare i valori da i-k a i+k

La formula base che useremo sarà questa:

$$B[i] = \sum_{j=-k}^{k} A[i+j] \cdot C[j+k]$$

Facciamo una piccola discussione:

- l'indice i indica la posizione di output ovvero l'elemento B che stiamo calcolando
- L'indice j scorre da -k a +k cioè tutto il raggio del filtro
- Il termine A[i+j] prende i valori d A attorno ad i
- Il termine C[j+k] prende i valori del filtro (con +k perché gli indici di C partono da 0)

Questa formula che abbiamo visto ora non è altro che quella di prima del caso finito (versione generale) invece questa qui è una versione centrata a indice 0, identifichiamo i ruoli:

A vettore di dati, ovvero quello che subisce la convoluzione, C il filtro e B il risultato.

Invece di usare m, qui usiamo j. Ora A gioca il ruolo di f, C il ruolo di g.

i è l'indice Array A(corrisponde a n)

J indice secondo array (ovvero la traslazione di A, serve appunto per scorrere intorno ad I)

C è il filtro centrato, non rovesciato (lo vediamo dal fatto che è $j + k$ e non $j - k$, questo significa che C cresce nella stessa direzione dell'indica di A, il +k serve solo a spostare l'origine del filtro, ovvero il suo centro.)

K = serve a spostare l'origine degli indici del filtro C. Se non usassimo K otterremo degli indici di C negativi.

Facciamo un esempio veloce:

Nelle slide viene mostrato $B[i] = \sum_{j=k}^{2k+1} A[i+j] \cdot C[j+k]$ per semplicità di calcoli

questo tenendo conto del funzionamento che ipotizzando k= 2 avremmo j che va da 2 a 5 che è molto più facile da calcolare che tenere conto di -2 a 2 nell'utilizzo semplice della formula:

$$B[i] = \sum_{j=-k}^{k} A[i+j] \cdot C[j+k]$$

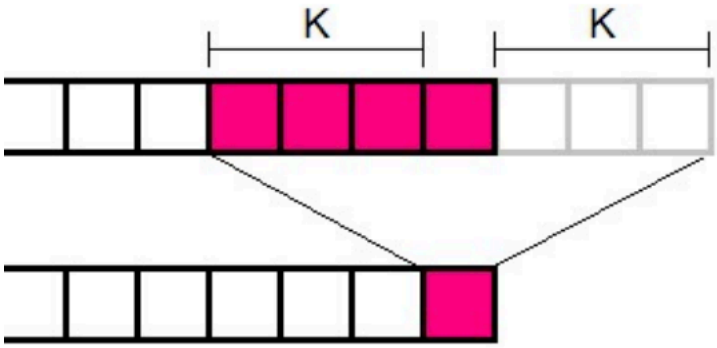
Che nel nostro sarà:

$$B[i] = \sum_{j=-2}^{2} A[i+j] \cdot C[j+k]$$

Prendiamo anche il codice:

```
Begin
        for i=0; i<N; i++ do
                for (j=-k; j<=k; j++) do
                        if(I+j>=0 && i+j <N) do
                                B[i] = B[i] + A[i+j]*C[j+k]
                        endif
                endfor
        endfor
end
```

Adesso l'if ha lo scopo di impedire che vengano prese in considerazione i casi in cui il filtro si trova al di fuori dell'array, il primo blocca dietro e il second blocca avanti:



Prendiamo N=10 con k= 2

B[0]=1·3+2·2+3·1=3+4+3=10

B[1]=1·2+2·3+3·2+4·1=2+6+6+4=18

B[2]=1·1+2·2+3·3+4·2+5·1=1+4+9+8+5=27

B[3]=2·1+3·2+4·3+5·2+6·1=2+6+12+10+6=36

B[4]=3·1+4·2+5·3+6·2+7·1=3+8+15+12+7=45

B[5]=4·1+5·2+6·3+7·2+8·1=4+10+18+14+8=54

B[6]=5·1+6·2+7·3+8·2+9·1=5+12+21+16+9=63

B[7]=6·1+7·2+8·3+9·2+10·1=6+14+24+18+10=72

B[8]=7·1+8·2+9·3+10·2=7+16+27+20=70

B[9]=8·1+9·2+10·3=8+18+30=56

Otterremo poi infinite l'array finale [10,18,27,36,45,54,63,72,70,56]

Vengono quindi eseguiti N prodotti scalari di lunghezza 2k+1 = 5

La complessità sequenziale di tale algoritmo è pari a N volte 2L-1 operazioni

Dove L = 2k+1

$$T_1(N,k) = N[2L-1] = N[2(2k+1)-1] = N[4k+2-1] = N[4k+1]$$

Abbiamo N/p convocazioni locali quindi : $T_p(N,k) = \frac{N}{p} = \frac{N[4k+1]}{p}$

$$S_p = \frac{N[4k+1]}{\frac{N[4k+1]}{p}} = p$$

$$O_h = 0$$

$$E_p = 1$$

Se N non divisibile per p?

Facciamo finta che tutti avessero un vettore di lunghezza 9 da elaborare quindi:

$int(\frac{n}{p} + 1)$ convocazioni di lunghezza 2k+1

Lezione 18 - Decomposizione 2D - Matrici full parallel

Viene ripassato il concetto di decomposizione, con funzionale e dominio compreso il ripartire dal modello numerico.

è un modello matematico discretizzato, cioè suddiviso in N passi numerici (h = 1,…,N).

L'obiettivo è distribuire questi N passi su più unità processanti (CPU o core).

- **Decomposizione funzionale**
  Ogni unità processante esegue **un passo diverso** del modello.Dove ogni processore fa un'operazione differente su parti distinte del problema.

- **Decomposizione del dominio Dove t**utte le unità processanti eseguono **la stessa operazione**, ma su **sottoinsiemi diversi di dati**.

- **Combinazione delle due**: quando conviene, si può usare una **decomposizione ibrida**, che mescola le due strategie precedenti.

Il modello $M_h(P)$ viene "scomposto" in **sotto-task paralleli**.
L'obiettivo è:

- suddividere i task in **sotto-task uguali**,
- processarli **contemporaneamente**,
- ridurre **al minimo la collezione dei risultati locali** (cioè la fase di aggregazione o sincronizzazione).

In figura vengono rappresentati vari moduli $M_1(P), M_2(P), \ldots M_N(P)$:
ognuno rappresenta una **parte del modello numerico** assegnata a un processore.

La decomposizione può anche essere **ricorsiva**: un modulo $M_2(P)$ può essere ulteriormente diviso in $M_{2.1}(P), M_{2.2}(P), \ldots M_{2.NP}(P)$. Questo rappresenta la **decomposizione del dominio** a livelli più fini.

Decomposizione di matrici

Vediamo come primo concetto il prodotto di uno scalare con una matrice di grandi dimensioni.

Prendiamo una matrice A di dimensione N*N e uno scalare β

Quello che noi ci aspettiamo come output sarà $C = c_{i,j} = \beta * a_{i,j}$ con

$i = 0,..,N-1$ e $j = 0,..,N-1$

Metodo Sequenziale

Vengono eseguite N * N moltiplicazioni una dopo l'altra:

```
for i = 0 to N-1 do
   for j = 0 to N-1 do
      c[i,j] := β * a[i,j];
```

Metodo Parallelo

Come già sappiamo Nel modello **MIMD (Multiple Instruction Multiple Data)** a **memoria condivisa**, abbiamo più CPU che possono accedere contemporaneamente alla stessa area di memoria.

Se disponiamo di `np` unità processanti, possiamo **decomporre il problema** in `np` sottoproblemi, ciascuno di dimensione $\frac{N}{np}$

- Matrice N*N → divisa in `np` blocchi.
- Ogni CPU elabora il proprio blocco.

Per affrontare il problema "scalare × matrice" in modo parallelo, si procede in tre passi:

1. Partizionamento della matrice A in blocchi.
2. Riformulazione dell'algoritmo sequenziale "a blocchi".
3. Parallelizzazione dell'algoritmo "a blocchi".

Ogni blocco della matrice che non necessita dei risultati degli altri può essere calcolato indipendentemente da ciascun processore.

1 Strategia

L'idea è quella di suddividere la matrice A in blocchi di righe e assegniamo il calcolo dello scalare per questi blocchi di righe a ogni CPU.

I risultati C_i vengono poi riuniti nella memoria shared per formare C. **Abbiamo un parallelismo completo.**

Prendiamo ad esempio N = 6 e p=2

Ogni processore calcolare 3 righe della matrice. Non abbiamo nessuna dipendenza tra i 2 blocchi di righe e ogni core lavora contemporaneamente (Full parallel)

$$A = \begin{pmatrix} a_{00} & a_{01} & a_{02} & a_{03} & a_{04} & a_{05} \\ a_{10} & a_{11} & a_{12} & a_{13} & a_{14} & a_{15} \\ a_{20} & a_{21} & a_{22} & a_{23} & a_{24} & a_{25} \\ a_{30} & a_{31} & a_{32} & a_{33} & a_{34} & a_{35} \\ a_{40} & a_{41} & a_{42} & a_{43} & a_{44} & a_{45} \\ a_{50} & a_{51} & a_{52} & a_{53} & a_{54} & a_{55} \end{pmatrix} \cdot \beta \quad \begin{matrix} c_0 \\ \\ \\ c_1 \\ \\ \end{matrix}$$

2 Strategia

Suddividiamo la matrice A in blocchi di colonne e assegniamo il calcolo dello scalare per questi blocchi di colonne a ogni CPU

Anche qui abbiamo per lo stesso motivo del procedente parallelismo completo.

Prendiamo sempre lo stesso esempio:

$$A = \begin{pmatrix} a_{00} & a_{01} & a_{02} & a_{03} & a_{04} & a_{05} \\ a_{10} & a_{11} & a_{12} & a_{13} & a_{14} & a_{15} \\ a_{20} & a_{21} & a_{22} & a_{23} & a_{24} & a_{25} \\ a_{30} & a_{31} & a_{32} & a_{33} & a_{34} & a_{35} \\ a_{40} & a_{41} & a_{42} & a_{43} & a_{44} & a_{45} \\ a_{50} & a_{51} & a_{52} & a_{53} & a_{54} & a_{55} \end{pmatrix} \begin{matrix} \\ \\ \\ \\ \\ \\ \end{matrix} \begin{matrix} \\ \\ \\ \\ \\ \cdot \beta \end{matrix} \begin{matrix} \\ \\ \\ \\ \\ \cdot \beta \end{matrix}$$

c_0 c_1

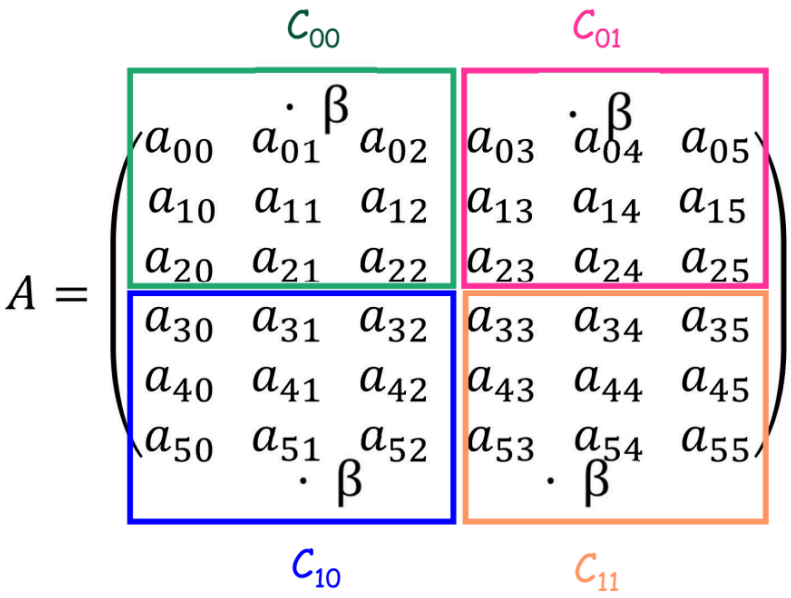
3 Strategia

L'idea è suddividere A per blocchi di righe e colonne. Cioè una grigia di sottoblocchi. Dobbiamo dividere la matrice in q blocchi di righe e p blocchi di colonne. Se abbiamo ad esempio come nell'immagine N=6 e q*p = 2*2 avremo 4 sottoblocchi che verranno assegnati 1 ad ogni core.

Se abbiamo $q * p$ core allora la matrice viene partizionata in $q * p$ blocchi.

Ogni sottoblocco viene elaborato da una CPU

Il risultato e nella memoria condivisa. **Anche questo come tutti accedere alla memoria globale su dati differenti quindi niente conflitti o sincronizzazione.**



il problema è FULL PARALLEL poiché tutte le strategie sono completamente parallelizzabili. Non abbiamo di conseguenza una collezione di risultati.

Lezione 19 - Valutazioni per matrici full parallel

Prendiamo una matrice A di dimensioni N*N aspettandoci un output pro a
$C = c_{ij} = \beta \cdot a_{ij}$

Ogni elemento della matrice di output si ottiene moltiplicando lo scalare per l'elemento corrispondente della matrice d'ingresso.

Se prendiamo ad esempio $\beta = 2$

A = {[1,3][2,4]} => C = {[2,6][4,8]} Ogni prodotto è indipendente e perfettamente parallelizzabile.

---

1 Strategia

Suddividiamo la matrice A in blocchi di righe, uno per ogni processore. Prendiamo un esempio con p = 3.

Complessità sequenziale

Abbiamo N * N prodotti e di conseguenza abbiamo un $T_1(N^2) = N^2$.

Questo perché essendo la nostra una matrice quadrata a cui vogliamo moltiplicare uno scalare per ogni elemento della matrice A, dobbiamo eseguire una moltiplicazione per ciascun elemento. Quindi
$N_{elementi} = N(righe) \cdot N(colonne) = N^2$

Complessità parallela

Ogni core calcola appunto il proprio blocco quindi la matrice totale ha N righe che divise per p processori ci porta ad avere $\frac{N}{p}$. Ogni riga poi ha N colonne, quindi la dimensione del blocco locale è $\frac{N}{p} * N = \frac{N^2}{p}$

Questo significa che ogni processore riceve una sotto-matrice con N/p righe e N colonne.

Quindi se $T_1(N^2)$ è il tempo per fare le operazioni in sequenza allora con p processori il lavoro sarà $\frac{N^2}{p}$. La N rappresenta le colonne e N/p le righe assegnate a ciascun processore.

Esempio rapido per capire:

Immagina:

- N=8 (matrice 8×8)
- p=4 processori

→ Ogni processore riceve 48 =2 righe.
→ Dimensione locale: 2×8.
→ Elementi = 2×8=16.
→ Totale elementi globali: 16×4=64=8×8.

Detto ciò $T_p(N^2) = \frac{N^2}{p} = \frac{64}{4} = 16$

Speed-Up

$$S_p = \frac{T_1(N^2)}{T_p(N^2)} = \frac{N^2}{\frac{N^2}{p}} = p$$

OverHead

$$O_h = pT_p(N^2) - T_1(N^2) = p[\frac{N^2}{p} - N^2 = 0$$

Efficienza

$$E_p = 1$$

---

2 Strategia

Praticamente stesso identico comportamento della 1 strategia, stessi calcoli.

---

3 Strategia

In questa strategia la matrice A viene divisa in una griglia di blocchi, sia per righe sia per colonne.

Immagino di dividere le righe in q gruppi e le colonne i p gruppi.

Otteniamo una griglia di q*p sotto matrici.

La matrice ha N righe totali che se le divido in q parti uguale ogni blocco avrà $\frac{N}{q}$

righe.

Ha ovviamente anche N colonne totali, che se divise in p parti uguale ogni blocco ha $\frac{N}{p}$ colonne.

Ogni sottomatòice è assegnata a un processore diverso. Il numero totali di processori sarà:

$$P = q \cdot p$$

Questo è perché in pratica prendiamo il caso di avere 4 processori, questi vengono divisi 2 per le colonne e 2 per le righe, in pratica dobbiamo vedere una cosa del genere:

P_0	P_1
P_2	P_3

Vediamo che ogni processore ha il suo blocco assegnato, per questo motivo abbiamo $\frac{N}{q}$ X $\frac{N}{p}$.

Ogni processore dovrà quindi calcolare $\frac{N^2}{qp}$ operazioni.

Quindi ricapitolando:

In sequenziale abbiamo sempre un $T_1(N^2) = N^2$

Abbiamo poi $T_{qp}(N^2) = \frac{N^2}{qp}$ prodotti locali

Speed-Up

$$S_p = \frac{T_1(N^2)}{T_{qp}(N^2)} = \frac{N^2}{\frac{N^2}{qp}} = qp$$

OverHead

$$O_h = qpT_{qp}(N^2) - T_1(N^2) = qp[\frac{N^2}{qp} - N^2 = 0$$

Efficienza

$$E_p = 1$$

Isoefficienza

$$I = 0/0$$

È una forma indeterminata e per convenzione I può assumere qualsiasi valore. Questo avviene perché gli algoritmi full parallelo sono naturalmente scalabili. Questo vale per tutte e 3 le strategie.

Quando p diverso a q

Ware Amdahl 1 Strategia

Caratterizziamo ora lo speed up usando Ware Amdahl

Prendiamo p = 8 e A = 32x32

In sequenziale abbiamo 32x32 =1024 prodotti

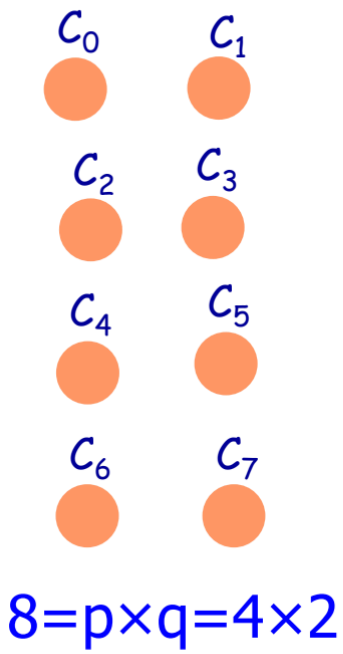
Prodotti parziali: Ogni processore esegue un totale di 4 x 32 =128

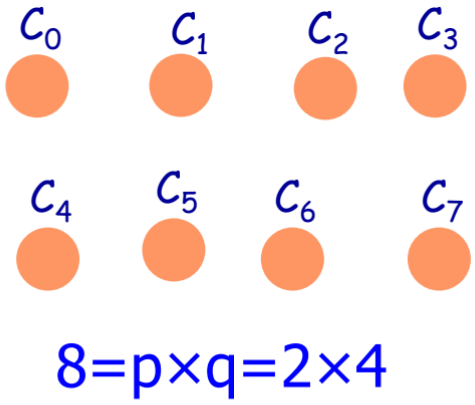
Di questi 128 prodotti vengono eseguiti in parallelo 128x8 =1024 quindi tutti.

$(1 - a) = 1024/1024$

Ware Amdahl 2 Strategia

Stessa identica situazione infatti:





Ware Amdahl 3 Strategia

Prendiamo p = 8 e A = 32 x 32

P.= 4 righe e q =2 colonne. (Ricordiamo che p = q*p che fa 8 in questo caso)

Calcoliamo i prodotti parziali = $\frac{32}{4} \cdot \frac{32}{2} = 128$ che ci porterà anche qui a

$(1 - a) = 1024/1024$

Sarà la stessa identica cosa se avessimo impostato p = 2 righe e q = 4 colonne.

P=Q=2

P = 4 e A=32x32

Somme parziali = $\frac{N}{p} * N = \frac{32}{4} * 32 = 8$(righe) * 32 = 256 prodotti

Che ci riporta a $(1 - a) = 1024/1024$

Oltre ai calcoli la differenza non c'è.

Abbiamo sempre Ware Amdalh pari a 1.

Osservazioni finali

Tutte le strategie sono ugualmente naturalmente scalabili

Finche le matrici sono quadrate non c'è preferenza nella scelta fra 1 e 2 strategia

Se q è diverso da p non c'è differenza fra nessuna delle 3.

Nella terza però quando q = p abbiamo il denominatore dello speed-up c'è un quadrato.

Lezione 21 - Matrice * Vettore

Comunemente chiamate ance matrici di NxM e le dimensioni non sono esattamente divisibili per p o q.

Prodotto Matrice x Vettore

$y = A \cdot x$ con $A \in \mathbb{R}^{n*n}$ $x \in \mathbb{R}^n$ $y \in \mathbb{R}^n$

Algoritmo Sequenziale

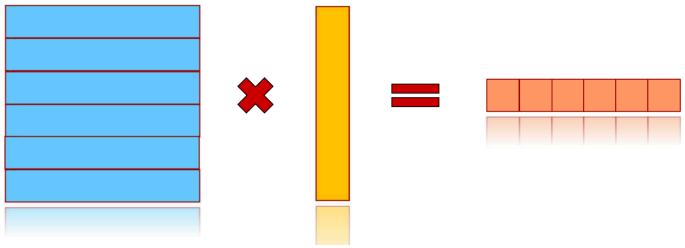
```
for i = 0 to n-1 do
   yi = 0
   for j = 0 to n-1 do
      yi = yi + aij * xj
```

```
    endfor
endfor
```

Ogni elemento y_i è ottenuto come prodotto scalare tra la riga i-esima della matrice A e il vettore x.

In un calcolatore sequenziale, le componenti di y vengono determinate una dopo l'altra cioè in modo ordinato, prima y_0 poi y_1 fino a y_{n-1}

Le componenti di y sono
calcolate effettuando i
prodotti scalari di
ciascuna riga di A per il vettore x



I prodotti scalari possono essere calcolati in maniera indipendente l'uno dall'altro.

Ogni y_i è indipendente dagli altri, infatti ogni riga di A usa sempre tutto x, ma nessuna riga interferisce con l'altra.

Questo rende il problema altamente parallelizzabile, poiché il lavoro si può dividere per righe, per colonne o per blocchi.

La strategia che prendiamo è:

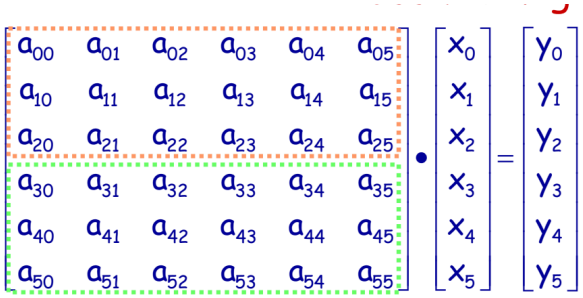
- La matrice A è suddivisa in blocchi
- Il vettore x in sotto-vettore corrispondenti.

Il calcolo di y viene riscatto come algoritmo a blocchi, dove ciascun processore calcola la parte di y associata al proprio blocco di A.

---

1 Strategia

Qui suddividiamo il lavoro sulla matrice A per blocchi di righe.



Ogni processore riceve un insieme di righe della matrice A e calcola le corrispondenti componenti del vettore y.

Prendiamo il caso di avere 2 core come dell'esempio:

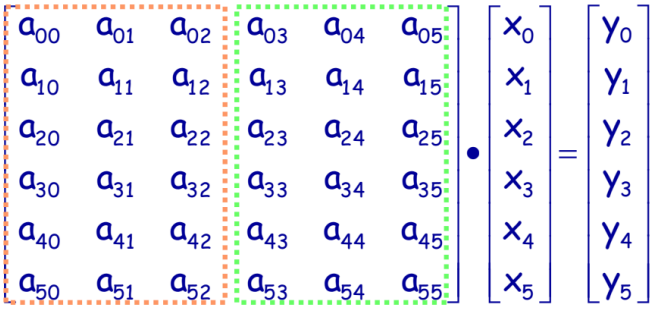
- Il core 0 calcola le prime 3 righe di A. Quindi valori y_0, y_1, y_2
- Il core 1 le restanti 3

Anche questa è una strategia full parallel

---

2 Strategia

Qui suddividiamo il lavoro sulla matrice A per blocchi di colonne.



Ogni core riceve un blocco di A, riceve poi la corrispondente parte del vettore x e calcola un risultato parziale per poi infine effettuare una somma finale dei risultati per ottenere y:

Stesso esempio di prima:

- Core 0 calcola le prime 3 colonne y_0, y_1, y_2
- Core 1 il restante

$$\begin{bmatrix} a_{03} & a_{04} & a_{05} \\ a_{13} & a_{14} & a_{15} \\ a_{23} & a_{24} & a_{25} \\ a_{33} & a_{34} & a_{35} \\ a_{43} & a_{44} & a_{45} \\ a_{53} & a_{54} & a_{55} \end{bmatrix} \cdot \begin{bmatrix} x_3 \\ x_4 \\ x_5 \end{bmatrix} = \begin{bmatrix} s_0 \\ s_1 \\ s_2 \\ s_3 \\ s_4 \\ s_5 \end{bmatrix} \quad \begin{bmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \\ a_{20} & a_{21} & a_{22} \\ a_{30} & a_{31} & a_{32} \\ a_{40} & a_{41} & a_{42} \\ a_{50} & a_{51} & a_{52} \end{bmatrix} \cdot \begin{bmatrix} x_0 \\ x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} r_0 \\ r_1 \\ r_2 \\ r_3 \\ r_4 \\ r_5 \end{bmatrix}$$

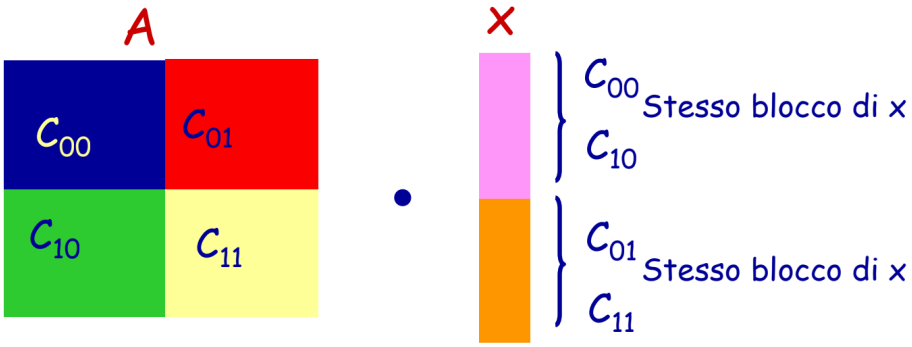
Core 1

I due vettori ottenuti da tale prodotto vengono poi sommati per ottenere il vettore y.

---

3 Strategia

Partizioniamo A in sottomatrici, ciascuna assegnata a un core. Ogni core elabora il proprio blocco e usa solo la parte x necessaria.



Ogni core lavora col pezzo di x che gli serve

Viene poi tutto calcolato in questo modo:

C_{00}

$$a_{00} \cdot x_0 + a_{01} \cdot x_1 + a_{02} \cdot x_2$$
$$a_{10} \cdot x_0 + a_{11} \cdot x_1 + a_{12} \cdot x_2$$
$$a_{20} \cdot x_0 + a_{21} \cdot x_1 + a_{22} \cdot x_2$$

C_{01}

$$a_{03} \cdot x_3 + a_{04} \cdot x_4 + a_{05} \cdot x_5$$
$$a_{13} \cdot x_3 + a_{14} \cdot x_4 + a_{15} \cdot x_5$$
$$a_{23} \cdot x_3 + a_{24} \cdot x_4 + a_{25} \cdot x_5$$

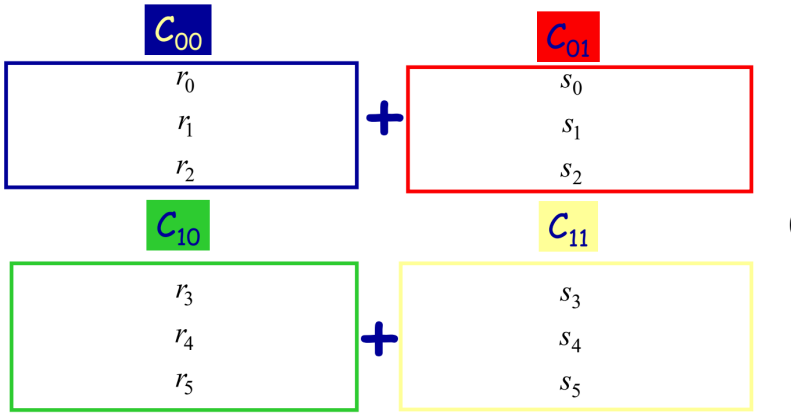
C_{10}

$$a_{30} \cdot x_0 + a_{31} \cdot x_1 + a_{32} \cdot x_2$$
$$a_{40} \cdot x_0 + a_{41} \cdot x_1 + a_{42} \cdot x_2$$
$$a_{50} \cdot x_0 + a_{51} \cdot x_1 + a_{52} \cdot x_2$$

C_{11}

$$a_{33} \cdot x_3 + a_{34} \cdot x_4 + a_{35} \cdot x_5$$
$$a_{43} \cdot x_3 + a_{44} \cdot x_4 + a_{45} \cdot x_5$$
$$a_{53} \cdot x_3 + a_{54} \cdot x_4 + a_{55} \cdot x_5$$

Le somme parziali ottenute esattamente come nella strategia numero 2 vengono poi sommate in parallelo



Lezione 23 - Valutazione Strategie

Si vuole calcolare

$$y = A \cdot x$$

Dove

- $A \in \mathbb{R}^{n*n}$ Una matrice N*M
- $x \in \mathbb{R}^{n}$ è un vettore colonna,
- $y \in \mathbb{R}^{n}$ è un vettore risultato

Per una singola riga di A servono M moltiplicazioni, una per ciascun termine $a_{ij} \cdot b_j$

Servono poi (M-1) addizioni per sommare tutti i prodotti.

Quindi per ogni riga il costo delle operazioni è pari a:

$$M + (M - 1)$$

Questo per 1 sola riga. Ora poiché la matrice sarà composta da N righe quello che facciamo è:

$$T_1(N * M) = N[M + (M - 1)] = N[2M - 1]$$

Questa è la complessità sequenziale, quindi su un singolo processore.

Prendiamo un esempio N (righe) = 4 e M (Colonne) = 3. Avremo per ogni riga 3 moltiplicazioni e 2 addizioni per un totale di 5 operazioni. Poi 4 * 5 = 20 operazioni.

1 Strategia

La **prima strategia** di parallelizzazione consiste nel dividere la **matrice A** in **blocchi di righe**, in modo che ciascun processore calcoli un sottoinsieme di prodotti scalari.

Se abbiamo:

- A di dimensione $\mathrm{N}\times\mathrm{M}$
- P processori (o core),

allora ogni processore gestirà:

$$\frac{N}{p} * M$$

Ricorda che ogni riga di A produce un elemento del vettore risultato y.
Quindi assegnare righe diverse a processori diversi **è la decomposizione più naturale**.

Ogni processore:

1. Prende il suo blocco di righe A_i ;
2. Calcola i corrispondenti y_i (prodotti scalari locali);
3. Scrive i risultati nella memoria condivisa.

Il vettore b è invece **interamente condiviso** tra tutti i processori (nessuna suddivisione, poiché serve a tutti).

$T_1(N * M) = N[2M - 1]$

Ogni processore esegue $\frac{N}{p}$ prodotti scalari, ognuno lungo M:

$$T_p(N * M) = \frac{N}{p}[2M - 1]$$

Questo perché abbiamo visto già T_1, tutto quello che abbiamo fatto è dividere N per p poiché stiamo appunto passando da 1 processore a p.

Vediamo ora Speed-UP:

$$S_p(N * M) = \frac{N[2M - 1]}{\frac{N}{p}[2M - 1]} = p\frac{N[2M - 1]}{N[2M - 1]} = p$$

$$O_h = pT_p(N*M) - T_1(N*M) = p(\frac{N}{p}[2M-1]) - N[2M-1] = 0$$

$$E_p(N*M) = \frac{S_p(N*M)}{p} = \frac{p}{p} = 1$$

Isoefficienza ci riporta una forma pari a 0/0 che è indeterminata. Ovvero per convenzione l'isoefficienza è posta pari ad infinito ovvero posso usare qualunque costante moltiplicativa per calcolare n_1.

Ware - Amdahl

Ora, per calcolare W-A la prima domanda che mi devo fare è se per questa strategia posso posso esattamente distinguere la parte parallela e la parte sequenziale.

In questo caso è SI quindi $S_p = \frac{1}{a + \frac{1-a}{p}}$ che è la formula.

Quindi adesso sappiamo già le nostre operazioni sequenziali, ovvero $T_1(N*M) = N[2M-1]$

Nella fase parallela eseguiremo le nostre $T_p(N*M) = \frac{N}{p}[2M-1]$

Da p processori contemporaneamente eseguiremo $p * \frac{N}{p}[2M-1]$ operazioni.

Quindi:

$(1-a) = p\frac{\frac{N}{p}[2M-1]}{N[2M-1]} = 1 \rightarrow \frac{1-a}{p} = \frac{1}{p}$ nella nostra parte sequenziale che non c'è significa che la nostra a= 0 quindi 1/p.

$$S_p = \frac{1}{\frac{1}{p}} = p$$

Seconda Strategia

A differenza della I strategia (per righe), qui dividiamo la **matrice A** per **colonne**.

- Matrice A: N×M
- Vettore b: M elementi

- Numero di processori: p

Ogni processore riceve **un blocco di colonne** di A e la parte corrispondente del vettore b.

Ogni processore calcola **un contributo parziale** al vettore risultato y.
Poi i risultati parziali vengono **sommati tra loro** nella memoria condivisa.

Quindi questa strategia ha due fasi:

1. Calcolo dei contributi parziali (tutti in parallelo);
2. **Somma finale dei vettori parziali** (sequenziale o parzialmente parallela).

Abbiamo quindi prima di tutte la divisione della matrice in colonne assegnate poi a un singolo processore che ne calcola il suo prodotto parziale. Stessa cosa viene fatta sul vettore b che viene diviso in base ai core.

Questo ci porta a delle dimensioni locali pari a:

A = $N * \frac{M}{p}$ e invece b = $\frac{M}{p}$

Analizziamo la prima A, allora se noi abbiamo p processori dividiamo la matrice A per colonne avremo quindi A = A0,A1,A_{p-1}. La matrice A ha M colonne in totale se la dividiamo in p blocchi ciascun blocco conterrà M/p colonne che ci porta alla dimensione locale di A.

Per il vettore b che viene diviso allo stesso modo, quindi dividiamo per p.

Tutti contemporaneamente eseguiranno $N[2\frac{M}{p} - 1]$ operazioni.

Ogni processore adesso riceve il blocco Ai che contiene M/p colonne e il sottovettore che contiene M/p elementi

E di conseguenza calcola il prodotto $A_i b^{(i)}$

Adesso A_i ha N righe e M/p colonne. Ogni moltiplicazione genera un vettore di N elementi.

Ogni riga che è formata da 1 solo elemento dovranno essere fatte M/p moltiplicazioni e M/p-addizioni (Prodotto scalare per avere quell'elemento)

Adesso il dubbio è come calcolare il risultato finale dato che dobbiamo fare le somme:

Potremmo usare le due strategie della somma, quindi:

- 1: Ogni core somma (uno alla volta) i vettori calcolati dagli altri core
  - In pratica il core C_0 legge le varie R_1,R_2 e li somma tutti a R_0. Il tempo totale di questa operazioni sarà p-1 fasi sequenziali per un totale di N*(p-1)
- 2: Applichiamo la riduzione binaria per $log_2(p)$ passi e un totale di N$log_2(p)$.

Speed-Up/Efficienza 2 Strategia SOMMA

$$T_p(N*M) = N[2\frac{M}{p} - 1] + Nlog_2(p)$$

$$S_p(N*M) = \frac{N[2M-1]}{N[\frac{2M}{p-1}] + Nlog_2(p)}$$

$$O_h = p(N[\frac{2M}{p-1} + NLog_2(p)) - N[2M-1] = N(-p + plog_2(p) + 1$$

$$E_p = p\frac{N[2M-1]}{N[\frac{2M}{p-1} + NLog_2(p)}$$

Isoefficienza

$$I = C = \frac{N_1(-p_1 + p_1log_2(p_1) + 1}{N_0(-p_0 + p_0log_2(p_0) + 1}$$

Cresce più lentamente in modo quasi lineare logaritmico. Questa strategia è più scalabile.

Speed-Up/Efficienza 1 Strategia SOMMA

$$T_p(N*M) = N[2\frac{M}{p} - 1] + (p-1)N$$

$$S_p(N*M) = \frac{N[2M-1]}{N[\frac{2M}{p-1}] + (p-1)N}$$

$$O_h = p(N[\frac{2M}{p-1} + (p-1)N) - N[2M-1] = N(-2p + p^2 + 1)$$

$$E_p = p\frac{N[2M-1]}{N[\frac{2M}{p-1}+(p-1)N}$$

Isoefficienza

$$I = C = \frac{N_1 N(-2p_1 + p_1^2 + 1)}{N_0(-2p_0 + p_0^2 + 1)}$$

Se fisso N_o e P_p la crescita richiesta per N_1 è quadratica rispetto a p_1

Ware-Amdahl

---

1 strategia di collezione

$S_p = \frac{1}{a + \frac{1-a}{p}}$ per la prima strategia

In sequenziale abbiamo T_1() = N[2M-1 operazioni di cui calcoli prodotti parziali N[2M/p -1] operazioni.

$1 - a = p\frac{N[2M/p-1]}{N[2M-1]} \rightarrow \frac{1-a}{p} = \frac{p}{p}\cdot\frac{2M/p-1}{2M-1}$ Per la fase parallela

Per la fase sequenziale $a = \frac{(p-1)N}{N[2M-1]} = \frac{p-1}{2M-1}$ questo perche in pratica aggiorno il vettore in P_0 facendo delle somme componente per componente.

Sostituzione nella formula: $S_p = \frac{2M-1}{2M/p+p-2}$

---

2 strategia di collezione

$$S_p = \frac{1}{a_1 + \sum_{k=2}^{p-1}\frac{a_k}{k} + \frac{a_p}{p}}$$

$$a_4 = \frac{4(M/2-1)}{2M-1}$$

$$a_2 = \frac{2}{2M-1}$$

$$a_1 = \frac{1}{2M - 1}$$

Dopo aver fatto la sostituzione: $S_p = \frac{2M - 1}{M/2 + 1}$

Lezione 25: Terza Strategia M*Vettore